

Resumen Prueba Practica

A continuacion los resúmenes de los ppts de las clases de ing software:

Contenidos:

- HTML + CSS
 - MVC + ROUTING
 - DISEÑO UML
 - TESTING
 - POLIMORFISMO Y LOOKUP
 - DISEÑO + ACOPLAMIENTO + COHESION
 - PATRONES DISEÑO COMPORTAMIENTO
 - PATRONES DISEÑO ESTRUCTURALES
 - PATRONES DISEÑO CREACIONALES
 - TESTING
 - ARQUITECTURA
 - DOCUMENTACION
 - ARQUITECTURA SOFTWARE
 - QUALITY ASSESMENT
-

9. HTML + CSS (Vistas)

HTML (HyperText Markup Language) es el lenguaje de marcado utilizado para estructurar y presentar contenido en la web. A continuación, se detallan algunas de las características más importantes de HTML, junto con ejemplos de código:

- **Estructuras:** HTML utiliza etiquetas para definir y organizar los distintos elementos y secciones de una página web.

- **Ejemplo:**

```
html
<!DOCTYPE html>
<html>
<head>
  <title>Mi Página Web</title>
</head>
<body>
  <header>
    <h1>Bienvenidos a Mi Página Web</h1>
  </header>
  <div>
    <p>Este es el contenido principal de la página.</p>
  </div>
  <footer>
    <p>&copy; 2024 Mi Página Web</p>
  </footer>
</body>
</html>
```

- **Enlaces:** Los enlaces se crean utilizando la etiqueta `<a>` con el atributo `href` para especificar la URL de destino.

- **Ejemplo:**

```
html
<a href="https://www.example.com">Visita Example</a>
```

- **Imágenes:** Las imágenes se incorporan utilizando la etiqueta `<img>` con el atributo `src` para especificar la ubicación de la imagen.

- **Ejemplo:**

```
html

```

- **Listas:** HTML permite crear diferentes tipos de listas:
 - **Ejemplo de lista ordenada:**

```
html
<ol>
  <li>Elemento 1</li>
  <li>Elemento 2</li>
  <li>Elemento 3</li>
</ol>
```

1. **Atributos de las etiquetas:** Los atributos proporcionan información adicional sobre los elementos HTML.

- **Ejemplo:**

```
html
<div class="contenedor">
  <p id="parrafoPrincipal">Este es un párrafo dentro de un contenedo
r.</p>
</div>
```

CSS (Cascading Style Sheets) es un lenguaje de hojas de estilo que se utiliza para describir la apariencia y el formato de un documento escrito en HTML.

A continuación, se destacan algunas características clave de CSS, junto con ejemplos de código:

- **Estilos externos:** Los estilos CSS pueden definirse en archivos separados con la extensión `.css` y enlazarse a los documentos HTML.
 - **Ejemplo:**

```
html
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" type="text/css" href="estilos.css">
```

```
<title>Mi Página Estilizada</title>
</head>
<body>
  <h1>Hola Mundo</h1>
  <p>Este es un párrafo estilizado.</p>
</body>
</html>
```

```
css
/* Archivo estilos.css */
body {
  background-color: #f0f0f0;
  font-family: Arial, sans-serif;
}

h1 {
  color: #333;
}

p {
  color: #666;
}
```

Atributos

- Los atributos CSS definen cómo se deben mostrar los elementos HTML.
- **Ejemplo:**

```
css
p {
  color: blue;
  font-size: 14px;
  background-color: yellow;
  margin: 10px;
  padding: 5px;
}
```

Clases

- Las reglas de estilo CSS pueden reutilizarse asignando la misma clase a varios elementos HTML.
- Ejemplo:**

html

```
<div class="myClass">Este es un div con clase reutilizable.</div>  
<p class="myClass">Este es un párrafo con la misma clase.</p>
```

css

```
.myClass {  
  color: red;  
  font-weight: bold;  
}
```

Estas características permiten a los desarrolladores crear y dar estilo a páginas web de manera eficiente y consistente.

Formulario

Un formulario HTML permite a los usuarios ingresar y enviar datos.

htmlCopiar código

```
<!DOCTYPE html>  
<html>  
<head>  
  <link rel="stylesheet" type="text/css" href="estilos.css">  
  <title>Ejemplo de Formulario</title>  
</head>  
<body>  
  <form action="/submit" method="post">  
    <label for="nombre">Nombre:</label><br>  
    <input type="text" id="nombre" name="nombre"><br><br>  
  
    <label for="email">Correo Electrónico:</label><br>  
    <input type="email" id="email" name="email"><br><br>  
  
    <label for="mensaje">Mensaje:</label><br>
```

```

<textarea id="mensaje" name="mensaje"></textarea><br><br>

<input type="submit" value="Enviar">
</form>
</body>
</html>

```

Combo Box (Menú Desplegable)

Un menú desplegable permite a los usuarios seleccionar una opción de una lista.

```

htmlCopiar código
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" type="text/css" href="estilos.css">
  <title>Ejemplo de Combo Box</title>
</head>
<body>
  <form action="/submit" method="post">
    <label for="opciones">Selecciona una opción:</label><br>
    <select id="opciones" name="opciones">
      <option value="opcion1">Opción 1</option>
      <option value="opcion2">Opción 2</option>
      <option value="opcion3">Opción 3</option>
    </select><br><br>

    <input type="submit" value="Enviar">
  </form>
</body>
</html>

```

10. MVC + ROUTING:

MVC

- Arquitectura que estructura la aplicación, permitiendo organizar la lógica, vistas, etc.
 - Modelo: Donde haremos todo nuestro manejo de datos con la base de datos.
 - Controlador: Maneja la lógica de negocio, requests de usuarios, y el flujo de llevar la data desde el modelo a la vista. (intermediario)
 - Vista: Responsable de lo que el usuario visualiza.
- Ejemplo
 - El **modelo** `Persona` generado en `app/models/persona.rb` :

```
class Persona < ApplicationRecord
  validates :nombre, presence: true
  validates :edad, numericality: { only_integer: true, greater_than_or_equal_to: 0 }
end
```

- El **controlador**: en `app/controllers/personas_controller.rb` , define las acciones CRUD

```
class PersonasController < ApplicationController
  before_action :set_persona, only: %i[show edit update destroy]

  def index
    @personas = Persona.all
  end

  def show
  end

  def new
    @persona = Persona.new
  end
end
```

```

def edit
end

def create
  @persona = Persona.new(persona_params)
  if @persona.save
    redirect_to @persona, notice: 'Persona fue creada exitosamente.'
  else
    render :new
  end
end

def update
  if @persona.update(persona_params)
    redirect_to @persona, notice: 'Persona fue actualizada exitosamente.'
  else
    render :edit
  end
end

def destroy
  @persona.destroy
  redirect_to personas_url, notice: 'Persona fue eliminada exitosamente.'
end

private

def set_persona
  @persona = Persona.find(params[:id])
end

def persona_params
  params.require(:persona).permit(:nombre, :edad)
end
end

```

- Crea las **vistas** en `app/views/personas/` :
- Si en el controlador no usamos el método "render" para devolver un HTTPResponse,
- Rails buscará un archivo .erb con el mismo nombre de la acción (ej: index), generará el HTML y enviará el HTTPResponse.
`app/views/pokemons/index.html.erb`

```
# --- Controlador ---
class PokemonController < ApplicationController

  def index
    @pokemons = Pokemon.all
  end
end

# --- index.html.erb ---
<% @pokemons.each do |pokemon| %>
  <tr>
    <td><%= pokemon.name %></td>
    <td><%= pokemon.pokemon_tupe %></td>
  <tr>
<& end %>
```

- Volviendo al ejemplo de la clase Persona:

`index.html.erb`

```
<h1>Personas</h1>

<table>
  <thead>
    <tr>
      <th>Nombre</th>
      <th>Edad</th>
      <th colspan="3"></th>
    </tr>
```

```

</thead>

<tbody>
  <% @personas.each do |persona| %>
    <tr>
      <td><%= persona.nombre %></td>
      <td><%= persona.edad %></td>
      <td><%= link_to 'Mostrar', persona %></td>
      <td><%= link_to 'Editar', edit_persona_path(persona) %></td>
      <td><%= link_to 'Eliminar', persona, method: :delete, data: { confir
m: '¿Estás seguro?' } %></td>
    </tr>
  <% end %>
</tbody>
</table>

<%= link_to 'Nueva Persona', new_persona_path %>

```

show.html.erb

```

<h1><%= @persona.nombre %></h1>
<p>
  <strong>Edad:</strong>
  <%= @persona.edad %>
</p>

<%= link_to 'Editar', edit_persona_path(@persona) %> |
<%= link_to 'Volver', personas_path %>

```

new.html.erb y edit.html.erb

Ambos formularios son similares, así que puedes usar un parcial para el formulario.

_form.html.erb

```

<%= form_with(model: persona, local: true) do |form| %>
  <% if persona.errors.any? %>

```

```

<div id="error_explanation">
  <h2><%= pluralize(persona.errors.count, "error") %> prohibited thi
s persona from being saved:</h2>
  <ul>
    <% persona.errors.full_messages.each do |message| %>
      <li><%= message %></li>
    <% end %>
  </ul>
</div>
<% end %>

<div class="field">
  <%= form.label :nombre %>
  <%= form.text_field :nombre %>
</div>

<div class="field">
  <%= form.label :edad %>
  <%= form.number_field :edad %>
</div>

<div class="actions">
  <%= form.submit %>
</div>
<% end %>

```

new.html.erb

```

<h1>Nueva Persona</h1>
<%= render 'form', persona: @persona %>
<%= link_to 'Volver', personas_path %>

```

edit.html.erb

```

<h1>Editar Persona</h1>
<%= render 'form', persona: @persona %>
<%= link_to 'Volver', personas_path %>

```

Formularios Rails

```
1 <%= form_with url: "/search", method: :get do |form| %>
2   <%= form.label :query, "Search for:" %>
3   <%= form.text_field :query %>
4   <%= form.submit "Search" %>
5 <% end %>
```

equivalente en HTML a:

```
1 <form action="/search" method="get" accept-charset="UTF-8" >
2   <label for="query">Search for:</label>
3   <input id="query" name="query" type="text" />
4   <input name="commit" type="submit"
5     value="Search" data-disable-with="Search" />
6 </form>
```

Search for:

Rails Form Helpers

```
<%= form.text_area :message, size: "70x5" %>
<%= form.hidden_field :parent_id, value: "foo" %>
<%= form.password_field :password %>
<%= form.number_field :price, in: 1.0..20.0, step: 0.5 %>
<%= form.range_field :discount, in: 1..100 %>
<%= form.date_field :born_on %>
```

```
<%= form.time_field :started_at %>
<%= form.datetime_local_field :graduation_day %>
<%= form.month_field :birthday_month %>
<%= form.week_field :birthday_week %>
<%= form.search_field :name %>
<%= form.email_field :address %>
<%= form.telephone_field :phone %>
<%= form.url_field :homepage %>
<%= form.color_field :favorite_color %>
```

Routing

```
Rails.application.routes.draw do
  resources :photos
end
```

Resources: Genera las rutas de la foto de manera automatica

Siguen el estandar de la web y de ruby

HTTP Verb	Path	Controller#Action	Used for
GET	/photos	photos#index	display a list of all photos
GET	/photos/new	photos#new	return an HTML form for creating a new photo
POST	/photos	photos#create	create a new photo
GET	/photos/:id	photos#show	display a specific photo
GET	/photos/:id/edit	photos#edit	return an HTML form for editing a photo
PATCH/PUT	/photos/:id	photos#update	update a specific photo
DELETE	/photos/:id	photos#destroy	delete a specific photo

Redner

```
def update
  if @pokemon.update(pokemon_params)
    redirect_to @pokemon, notice: 'Pokemon was succesfully update'
  else
    render :edit
  end
end
```

- **Renderizar a otra vista** (diferente del nombre del metodo) → **Render: (vista)**
- **Redirect_to:** No solo renderiza la vista "show", ya que para renderizarla necesitara probablemente la variable @pokemon
- **Redirect_to: hace REFRESH del browser en la pagina "show"** usando @pokemon, por lo que se envía otro HTTPRequest

Otras formas de usar render:

1. render: edit
2. render action: edit
3. render "edit"
4. render "books/edit"
5. render template: "books/edit"

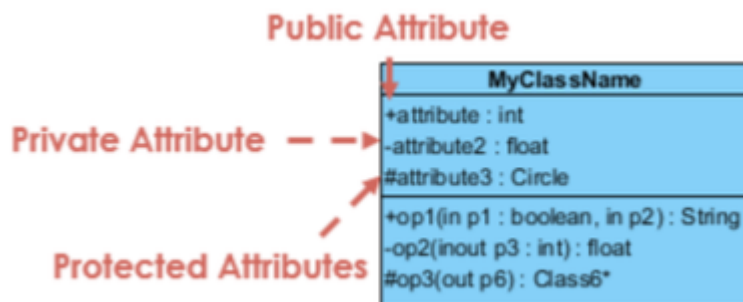
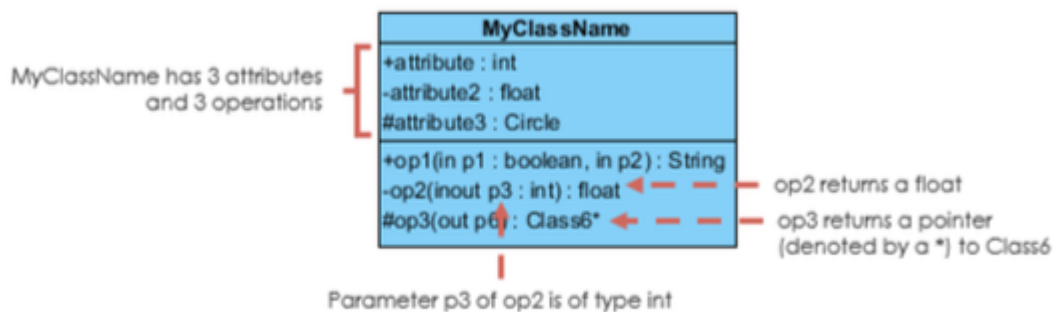
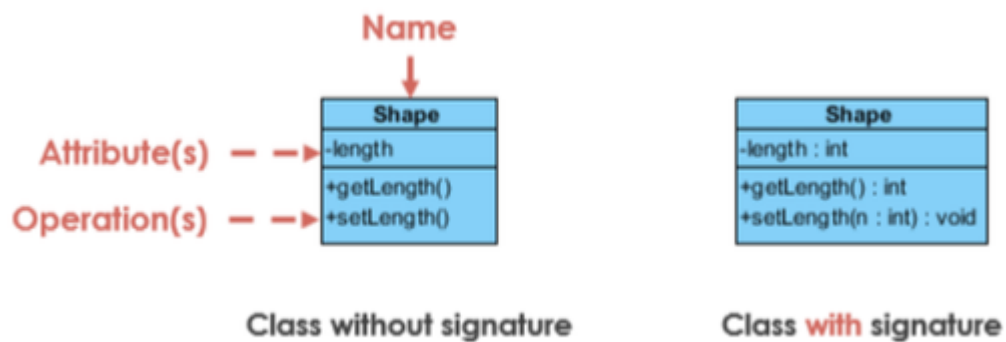
Todas son equivalentes

<https://guides.rubyonrails.org/routing.html>

https://guides.rubyonrails.org/layouts_and_rendering.html

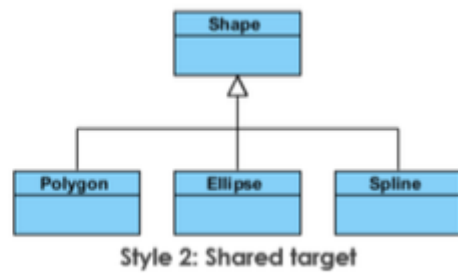
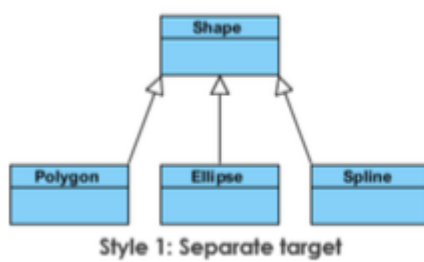
11. Diseño UML

Diagramas de clases



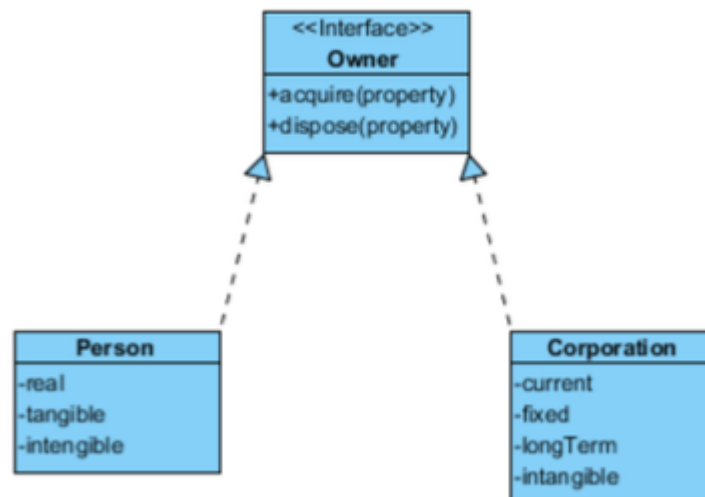
- **Público:** puede ser accedido desde cualquier clase del sistema.
- **Protegido:** solo puede ser accedido desde la misma clase y clases hijas.
- **Privado:** solo puede ser accedido desde la misma clase.

Herencia

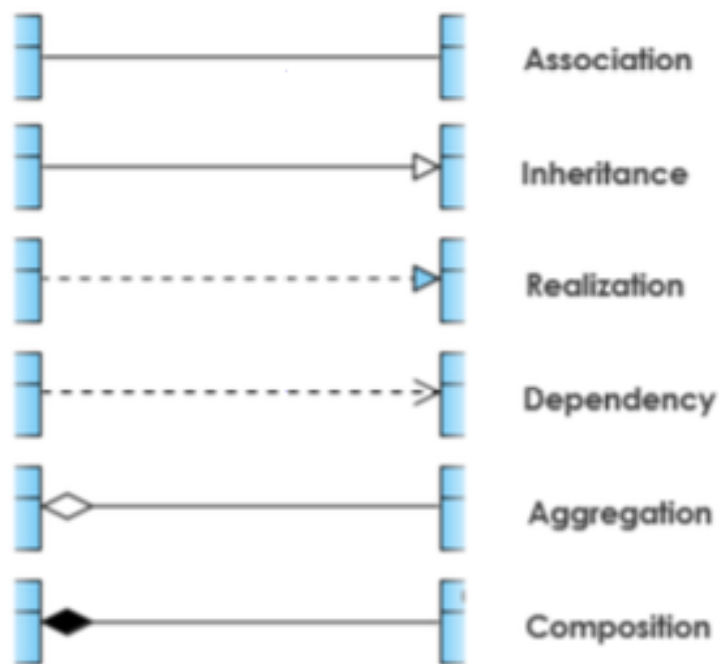


Se pueden utilizar cualquiera de los 2

Realizacion (Interfaz) → En ruby es un modulo



Relaciones entre clases



- **Association**: Un objeto de la clase tiene una referencia a un objeto de otra clase por mucho tiempo. (ej: atributos).
- **Herencia**: Una clase hereda de otra.
- **Realization**: Una clase implementa una interfaz o módulo.
- **Dependency**: un objeto de la clase tiene una referencia a otro objeto de otra clase de forma temporal (ej: variables temporales o argumentos)
- **Aggregation**: Un objeto de la clase A (rombo vacío) está compuesto por objetos de la clase B. Los objetos de B no son creados dentro de A.
- **Composition**: Un objeto de la clase A (rombo lleno) está compuesto por objetos de la clase B. Los objetos de B son creados dentro de B, creando dependencia. Un objeto derivado no podría existir sin su Principal

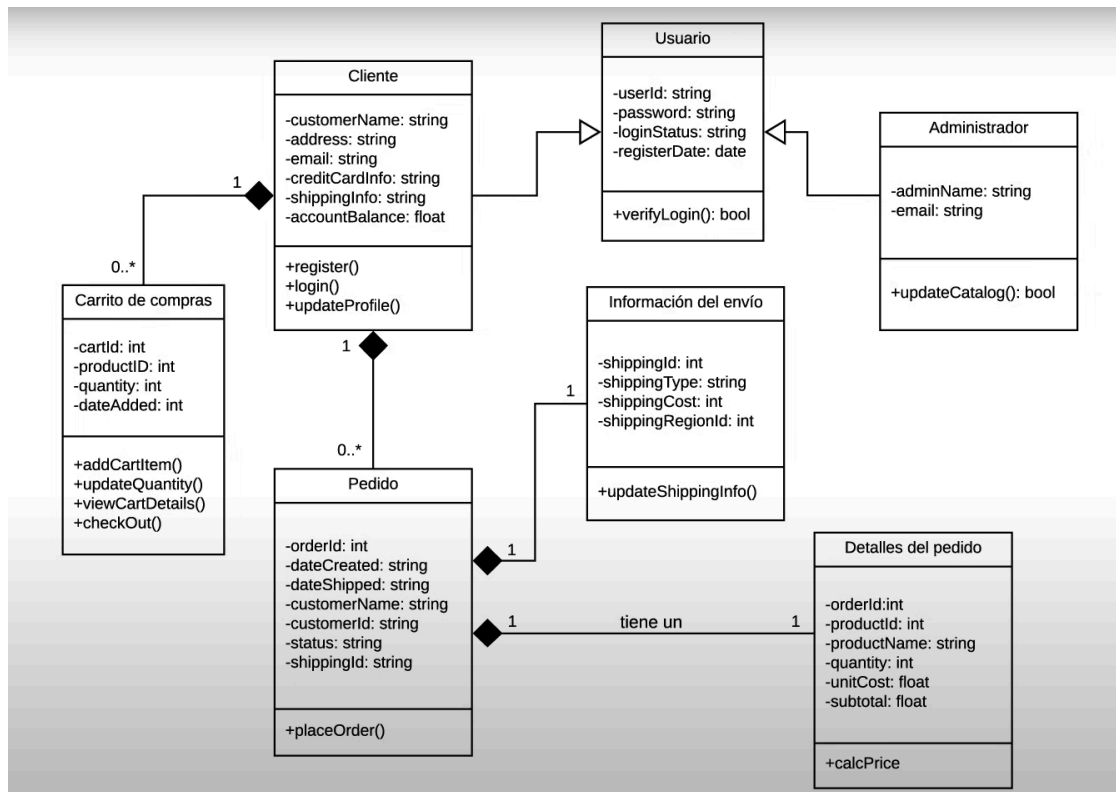
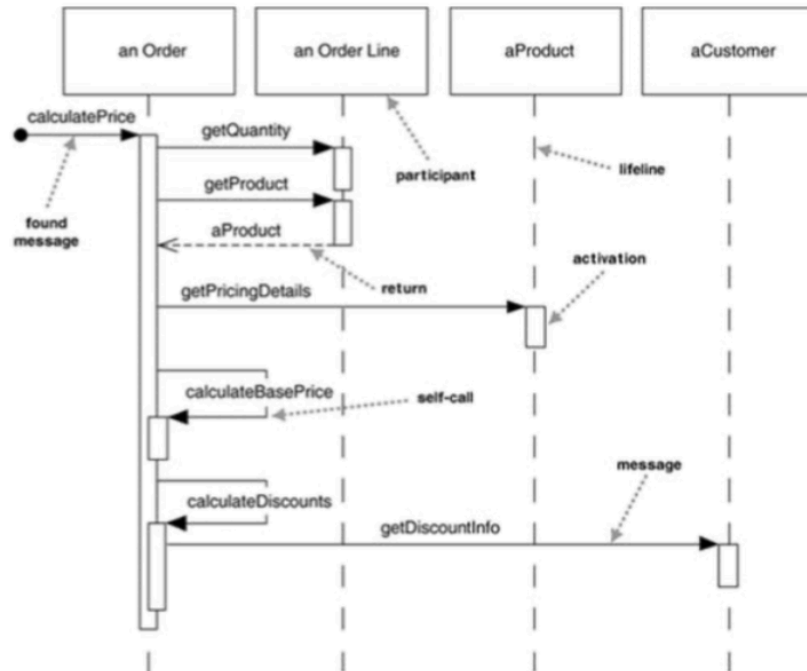


Diagrama de Secuencia

1. Cada Caja es la Instancia de una clase (objeto)
2. La línea punteada es la línea de vida de un objeto
3. Se lee de arriba hacia abajo



```

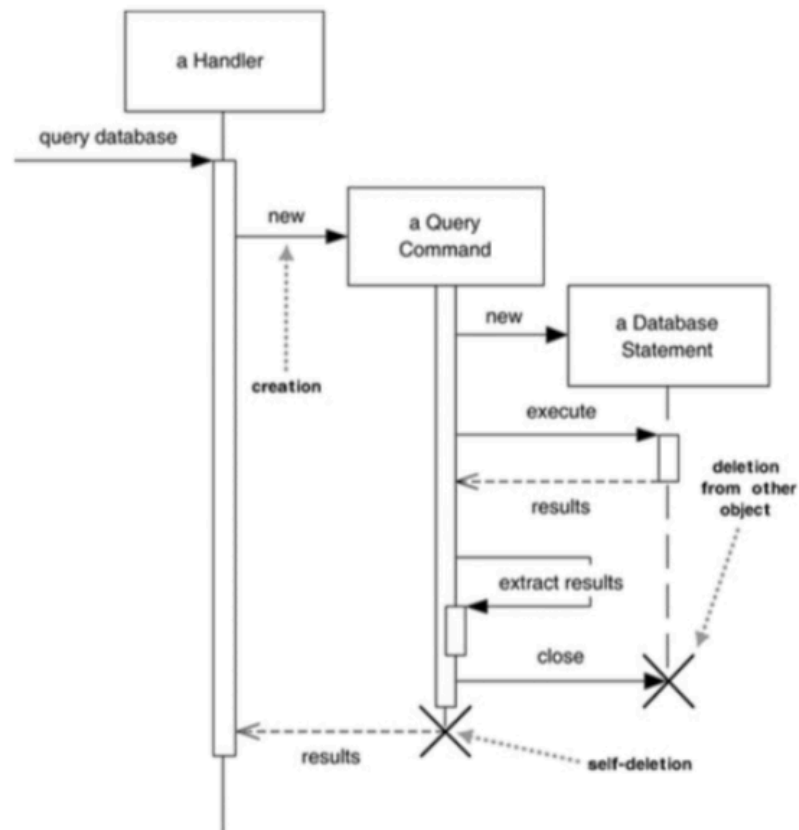
class Order
  def calculatePrice()

    a = orderLine.getQuantity()
    b = orderLine.getProduct()
    product.getPricingDetails()
    self calculateBasePrice()
    self calculateDiscount()
    customer.getDiscountInfo()

  end
end

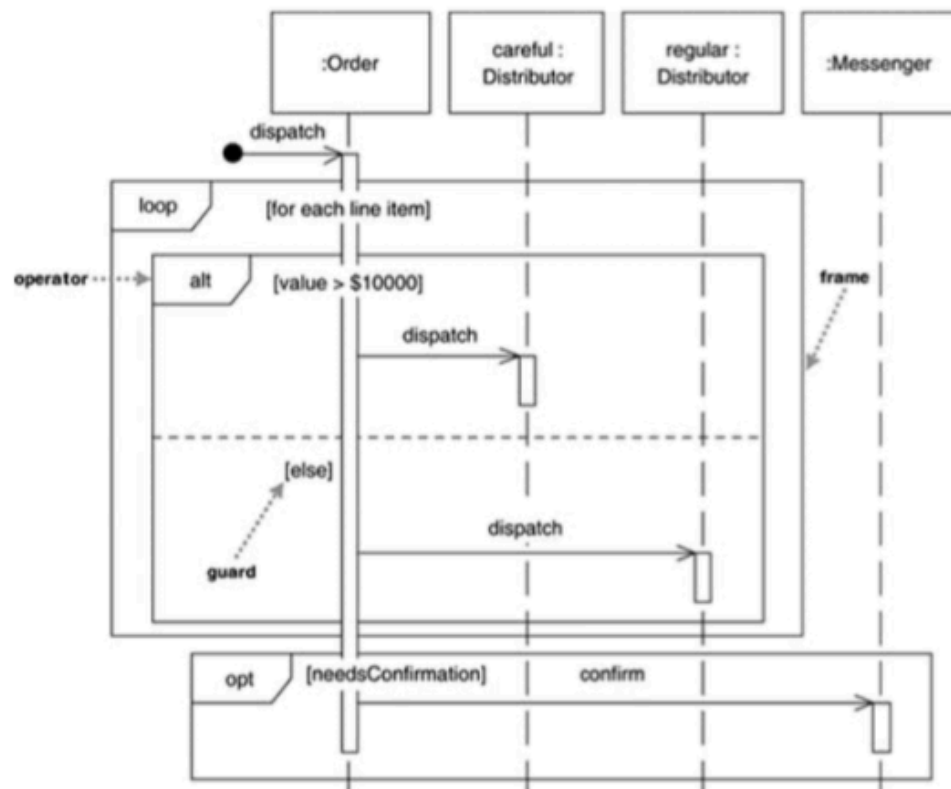
```

Creación y Eliminación de objetos



Iteraciones y Condicionales

- Es posible utilizar las dos notaciones "an Order" o "order1: Order".
- Alt es un "if-then-else", y entre corchetes va la condición.
- Loop hace referencia a un "for loop".
- El frame (marco) denota los mensajes que se llaman dentro del "if" o del "for".



12. Testing

Testing con rails

- **Unit testing**: En Rails, trata de verificar si las clases del modelo que creamos funcionan correctamente en isolación (por si solas, sin interacción de otras).
- **Integration testing**: En Rails, trata de verificar si los controladores creados en el proyecto están funcionando de manera correcta.
- **System testing**: En Rails, evalúa la funcionalidad del sistema como un todo. Es como abrir la página web, e interactuar con ella para ver que todo funcione. Se puede hacer manual o automática.

Unit test:

```

class Book < ApplicationRecord
  validates :title, presence: true
  validates :author, presence: true
  validates :year, presence: true
end

class BookTest < ActiveSupport::TestCase
  test "should not save without title" do
    @book = Book.new(title:"",author:"Juan P.", year:2023)
    result = @book.save
    assert_not result,"saved the title without title"
  end
end

```

Modelo

Test

```

class BookTest < ActiveSupport::TestCase
  test "No deberia guardarse sin titulo" do
    # Donde se crean objetos y datos necesarios
    @book = Book.new(title:"", author:"Juan P.", year:2023)

    # donde se realiza la accion a evaluar
    result = @book.save

    # verificacion, donde se evalua si el resultado esperado
    # assert o assert_not
    assert_not result, "saved the book without title"
  end
end

# EJECUCIÓN
rails test test/models/book_test.rb

```

Consideraciones:

- Rails tiene 3 bases de datos:
 - una para pruebas,
 - otra de producción,

- y otra para desarrollo. (esto se puede ver en config/database.yml).
- Rails inicializará una base de datos dedicada para ejecutar cada prueba, de esta forma cada test se ejecuta de forma separada y aislada.

Tipos de asserts

<code>assert(test, [msg])</code>	Ensures that <code>test</code> is true.
<code>assert_not(test, [msg])</code>	Ensures that <code>test</code> is false.
<code>assert_equal(expected, actual, [msg])</code>	Ensures that <code>expected == actual</code> is true.
<code>assert_not_equal(expected, actual, [msg])</code>	Ensures that <code>expected != actual</code> is true.
<code>assert_same(expected, actual, [msg])</code>	Ensures that <code>expected.equal?(actual)</code> is true.
<code>assert_not_same(expected, actual, [msg])</code>	Ensures that <code>expected.equal?(actual)</code> is false.
<code>assert_nil(obj, [msg])</code>	Ensures that <code>obj.nil?</code> is true.
<code>assert_not_nil(obj, [msg])</code>	Ensures that <code>obj.nil?</code> is false.
<code>assert_empty(obj, [msg])</code>	Ensures that <code>obj</code> is <code>empty?</code> .

→ Fixture

- Nos permite hacer ejemplos de datos
- En rails nos ayuda a crear objetos del modelo de manera facil
- `app/test/fixtures/nombre_del_modelo.yml`

```
jp1:
  author: Juan P. Sandoval
  title: Ingenieria de Software
  year: 2023

jp2:
  author: Juan P. Sandoval
  title: Testing
  year: 2022
```

Unit Test con Fixtures

```

class Book < ApplicationRecord
  validates :title, presence: true
  validates :author, presence: true
  validates :year, presence: true
end

class BookTest < ActiveSupport::TestCase
  test "should not save without title" do
    @book = books(:jp1)
    result = @book.save
    assert_not result, "saved the title without title"
  end
end

```

Integration Test

```

class BooksControllerTest <
  ActionDispatch::IntegrationTest
  test "should get index" do
    get books_url
    assert_response :success
  end

  test "should get new" do
    get new_book_url
    assert_response :success
  end
end

```

← Envía un request GET a la url "/books" y verifica que el response sea success.

← Envía un request GET a la url "/books/new" y verifica que el response sea success.

```

class BooksControllerTest < ActionDispatch::IntegrationTest
  test "should show book" do
    @book = Book.new(title:"Ing. Software",author:"Juan P.", year:2023)
    @book.save
    get book_url(@book)
    assert_response :success
  end
end

```

- Creamos un objeto libro en la base de datos
- Hacemos un GET request a /books/id, donde el id es el libro recién creado.
- Se verifica que el http response sea success.

- También se puede armar la ruta uno → get "/books/#{@book.id}"

SimpleCov

- Es una gema que nos permite ver cuántas líneas hemos testeado, y cuántas no.
- Para instalarlo hay que agregar la gema al archivo Gemfile, y ejecutar bundle install para que se instale:
 - gem 'simplecov', require: false, group :test

Líneas de código no cubiertas

```
25. # POST /books
26. def create
27.   @book = Book.new(book_params)
28.   if @book.save
29.     logger.info "redirect"
30.     redirect_to book_url(@book), notice: "Book was successfully created."
31.   else
32.     render :new, status: :unprocessable_entity
33.   end
34. end
```

Creamos un test para la creación exitosa de un libro, pero no consideramos cuando un libro no puede ser creado.

<https://guides.rubyonrails.org/>

13. Poliformismo y Lookup

Metodos:

- Nos permiten definir comportamientos de nuestras clases

```
class Person
  def greet
    "Hola"
  end
end
```

```
end

p1 = Person.new
puts p1.greet # Imprime Hola

# Otra forma sin guardar persona en una variable
puts Person.new.greet # Tambien imprime hola
```

```
# No es necesario instanciar para llamar a un metodo

class Developer
  def self.backend
    " I am a backend developer "
  end

  def frontend
    " I am a frontend developer "
  end
end

puts Developer.backend # Imprimira
puts Developer.frontend # dara error → tuvo que definirse con self
```

Visibilidad de metodos:

1. Privados: solo se acceden desde la misma clase
2. Public: - - -
3. Protected: - - -

Constructor

- El **new** es un método de clase que se ejecuta sobre la clase Person.
- Crea una instancia de la clase
- posteriormente llama al método initialize sobre el objeto recién creado.

```
class Persona

  def initialize
    puts "Creando Persona Nueva"
  end
end

Person.new # Imprimira lo del metodo → "Creando Persona Nueva"
```

Atributos:

- Los atributos en Ruby empiezan con "@"

```
class Persona
  def initialize(name)
    @name = name
  end

  def saludo(otra_persona)
    puts "Hola #{otra_persona}, me llamo #{@name}"
  end
end

pedro = Persona.new("Pedro")
pedro.saludo("Juan") # "Hola Juan, me llamo pedro"
```

Visibilidad Atributos:

- Por default los metodos son **PRIVADOS**, solo pueden ser accedidos desde la misma clase
- Para acceder a ellos se necesitan hacer "ACCESORES"

```
class Persona
  def initialize(name)
```

```

    @name = name
  end

  # Este metodo permite leer el atributo
  def name
    @name
  end

  # Este metodo permite modificar el atributo
  def name=(name)
    @name = name
  end

end

pedro = Persona.new("Pedro")
pedro.name= ("Esteban")
puts pedro.name

```

```

class Person
  attr_accessor :name, :gender # Crea los metodos name= y gender=
  attr_reader :age # No crea el metodo age=
end

```

Atributos compartidos por instancias distintas

@ → Atributo de Instancia

@@ → Atributo de Clase

```

class Person
  @@contador_personas = 0

  def initialize
    @@contador_personas += 1
  end

  # Metodo para visualizar el atributo

```

```

    def self.contador_personas
      @@contador_personas
    end
  end

  puts Person.contador_personas # 0
  Person.new
  puts Person.contador_personas # 1

```

- En ruby casi todo es Objeto → Tienen metodos con los cuales interactuar

```

s = String.new("Hola") # ⇒ "Hola"
s.length
a = Array.new # ⇒ []
a.push(s)
a.reverse

```

Herencia

Clase Padre → Clase Hija

```

class Figura
  attr_accessor: :stroke :fill
end

class Circulo < Figura
  attr_accessor: :radius # Ademas de stroke y fill
end

c1 = Circulo.new
c1.fill = "red"

```

Polimorfismo

Capacidad de un objeto de tomar otras formas

→ Duck Typing ("Si camina como pato y hace quack, entonces debe ser un pato) 

```
def pintar(figura, x, y)
  set_cordenadas(x, y)
  figura.draw

class Circle
  attr_accessor :radius

  def draw
    ...
  end

class Triangulo
  attr_accessor :base :altura
  def draw
    ...
  end

# Basicamente puedo llamar a pintar
# Le entrego cualquier clase (que tenga el metodo draw)
# pintar va a hacer draw del objeto entregado
```

Jerarquía de clases

```
# 3 niveles de jerarquía
class Figure
  ...

class Circle < Figure
  attr_accessor: :radius
  ...
```

```
class Cylinder < Circle
  attr_accessor: :length
```

Super

```
class Padre
  def initialize
    "Soy el padre"
  end
end

class Hijo
  def initialize
    super      # → super: ejecutara el Constructor (initialize) del Padre
  end
end
```

Herencia y Constructor

Este ejemplo: ejecuta el constructor padre mandando atributos necesarios

```
class Figure
  attr_accessor: :stroke, :fill
  #-----
  def initialize(stroke, fill)
    @stroke = stroke
    @fill = fill
  end
  #-----
end

class Circle < Figure
  attr_accessor: :radius

  def initialize(stroke, fill, radius)
```

```

        super(stroke, fill) # → Constructor del padre con sus respectivos atributos
        @radius = radius
    end
end

```

Override

- La sobreescritura es escribir un metodo existente en la clase padre, en la clase hija
- Tiene mismo nombre y argumentos que el metodo del padre
- ej: "to_s" : Metodo de Object (propio de ruby)

```

class Circle < Figure

    def initialize(radius)
        @radius = radius
    end

    def to_s
        "Es un circulo de radio #{@radius}"
    end

end

c = Circle.new(5)
puts c.to_s

```

Super y Override

- Con "super" podemos llamar metodo de la clase con el mismo nombre!

```

class Empleado
    def calcular_salario
        ...
    end
end

```

```
class Manager < Empleado
  def calcular_salario
    base_salario = super
    base_salario + @bonus
  end
end
```

Clases Abstracatas

- Es una clase incompleta, que se busca implementar en la hija

```
class Figure
  def print
    raise NotImplementedError
  end

  class Square < Figure
  end

  f = Figure.new
  f.print # → Error (No se usa la clase abstracta!)
```

LOOKUP

- Algoritmo basico de búsqueda
 1. Buscar método M en la lista de metodos de instancia dentro de la clase del objeto
 2. Si no lo encuentra, buscar metodo M en la clase padre Recursivamente
 3. Si luego de buscar en toda la jerarquía el metodo no se encuentra, se invoca a method_missing

SELF vs SUPER

- SELF: Busca el metodo DESDE la CLASE DEL OBJETO que recibe el mensaje
- SUPER: Busca el metodo DESDE la CLASE PADRE donde se encuentra la llamada a "super"

```
class A
  def foo
    1
  end

  def bar
    3
  end
end

class B < A
  def bar
    foo + super
  end
end

puts B.new.bar

# Imprimira → 4
```

Explicacion:

1. Se crea la instancia y llama a bar
2. BAR llama a FOO → como no esta en la clase, busca en su clase PADRE
3. Suma a "Super" → Definicion de BAR en la clase PADRE
4. $1 + 3 = 4$

```

class S
  def foo
    self.bar
    puts "S>>foo"
  end

  def bar
    puts "S>>bar"
  end
end

class A < S
  def foo
    super
    puts "A>>foo"
  end
end

class B < S
  def bar
    puts "B>>bar"
  end
end

class C < S
  def foo
    puts "C>>foo"
  end
end

S.new.foo # "S>>bar" "S>>foo"
A.new.foo # "S>>bar" "S>>foo" "A>>foo"
B.new.foo # "B>>bar" "S>>foo"
C.new.foo # "C>>foo"

```

14. Diseño Acoplamiento y Cohesión

Diseño

- Un buen diseño es facil de entender, modificar, testear y extender
- Se puede reutilizar
- Facil de Implementar

Heuristica Usada:

- Evitar código duplicado (code clones)
- Minimizar las dependencias entre clases (acoplamiento)
- Principio de Simple Responsabilidad (cohesion)
- Facilitar el mantenimiento y la extensibilidad.
 - Organizando el código para el cambio.
 - Aislando para el cambio.

Acoplamiento

- Se refiere a la **INTERACCION ENTRE CLASES** → Que tanto estan CONECTADAS
- + Diseño → - Acoplamiento
 - Buscamos tener clases muy independientes que se comuniquen atraves de pocos metodos bien definidos

Cohesión

- Se refiere a que tan BIEN un codigo MAPEA a una ENTIDAD
- + Diseño → + Cohesión
 - Alta cohesion → Cada clase/metodo/modulo es responsable de una tarea
 - Buenas clases → Alto nivel de cohesion

Ejemplo 1 (Metodos y datos entrelazados):

ShoppingCar.rb

```
1 require_relative 'item'
2 class ShoppingCar
3   def initialize
4     @items = []
5   end
6
7   def add(item)
8     @items.push item
9   end
10
11  def print
12    @items.each do |item|
13      puts "name: #{item.name}"
14      puts "quantity: #{item.quantity}"
15      puts "cost: #{item.totalCost}"
16    end
17  end
18 end
```

Item.rb

```
1 class Item
2   def initialize(item_name,item_quantity,item_cost)
3     @name = item_name
4     @quantity = item_quantity
5     @cost = item_cost
6   end
7   def name
8     @name
9   end
10  def quantity
11    @quantity
12  end
13  def totalCost
14    return @cost * @quantity
15  end
16 end
17
```

main.rb

```
1 require_relative "shopping_car"
2 require_relative "item"
3
4 car = ShoppingCar.new
5 car.add(Item.new("Pancito",5,200))
6 car.print
```

- Problemas:
 1. ShoppingCar depende de 2 atributos y un metodo de otra clase (Item)
 2. Aumenta el acoplamiento entre clases
 3. Rompe el encapsulamiento → Datos de los objetos deben estar lo mas ocultos posibles para evitar dependencias innecesarias

Codigo Mejorado:

ShoppingCar.rb

```
1 require_relative 'item'
2 class ShoppingCar
3   def initialize
4     @items = []
5   end
6
7   def add(item)
8     @items.push item
9   end
10
11  def print
12    @items.each do |item|
13      item.print
14    end
15  end
16 end
17
```

Item.rb

```
1 class Item
2   def initialize(item_name,item_quantity,item_cost)
3     @name = item_name
4     @quantity = item_quantity
5     @cost = item_cost
6   end
7   def print
8     puts "name: #{@name}"
9     puts "quantity: #{@quantity}"
10    puts "cost: #{self.totalCost}"
11  end
12  private:
13    def totalCost
14      return @cost * @quantity
15    end
16  end
17
```

main.rb

```
1 require_relative "shopping_car"
2 require_relative "item"
3
4 car = ShoppingCar.new
5 car.add(Item.new("Pancito",5,200))
6 car.print
```

**** Require_relative **** se utiliza para cargar archivos desde una ruta relativa al archivo que está ejecutando el código

```
proyecto/
├── main.rb
├── lib/
│   └── utilidades.rb
```

#ruby

require_relative 'lib/utilidades' # Carga el archivo utilidades.rb relativo al a

Ventajas:

1. ShoppingCar solo depende del metodo print → Disminuye las dependencias (acoplamineto)
2. Item tiene sus datos privados + metodo TotalCost es privado

Ejemplo 2 (Codigo Duplicado):

```

BookStore.rb
1 require_relative 'book'
2 class BookStore
3   def initialize
4     @books = []
5   end
6   def add(book)
7     @books.push(book)
8   end
9   def filterByAuthor(name)
10    @books.each do |book|
11      if book.author == name
12        book.print
13      end
14    end
15  end
16  def filterByTitle(title)
17    @books.each do |book|
18      if book.title == title
19        book.print
20      end
21    end
22  end
23 end

Book.rb
2 class Book
3   attr_reader :title
4   attr_reader :author
5   attr_reader :year
6   def initialize(title, author, year)
7     @title = title
8     @author = author
9     @year = year
10  end
11  def print
12    puts "#@title by #@author version #@year"
13  end
14 end

main.rb
1 require_relative 'book.rb'
2 require_relative 'book_store.rb'
3
4 store = BookStore.new
5 store.add(Book.new("Testing", "juampi", 2022))
6 store.add(Book.new("Ing. Software", "jaime", 2022))
7 store.add(Book.new("Testing", "rodrigo", 2021))
8
9 puts 'searching for jaime'
10 store.filterByAuthor("jaime")
11
12 puts 'searching for testing'
13 store.filterByTitle("Testing")

```

- BookStore repite código en los métodos "filterByTitle" y "filterByAuthor"
- Si se quiere añadir otro filtro, habría más código duplicado

Código Mejorado:

```

1 require_relative 'book'
2 class BookStore
3   def initialize
4     @books = []
5   end
6   def add(book)
7     @books.push(book)
8   end
9   def filter(strategy)
10    @books.each do |book|
11      if strategy.check(book)
12        book.print
13      end
14    end
15  end
16 end

1 class FilterStrategy
2   def check(book)
3     raise NotImplementedError
4   end
5 end

```

```

1 require_relative 'filter_strategy'
2
3 class ByTitle < FilterStrategy
4   def initialize(title)
5     @title = title
6   end
7
8   def check(book)
9     book.title == @title
10  end
11 end

```

```

1 require_relative 'filter_strategy'
2
3 class ByAuthor < FilterStrategy
4   def initialize(author)
5     @author = author
6   end
7
8   def check(book)
9     book.author == @author
10  end
11 end

```

- Ya no tendremos que modificar BookStore ni Book en caso de añadir mas filtros
- Solo se crea otra estrategia (clase nueva que herado de FilterStrategy)
- Esta *“Abierto para Extensiones, Cerrado para Modificaciones”*

Ejemplo 3 (Codigo Duplicado Entre Clases)

```

1 class Tea
2   def prepareRecipe
3     boilWater
4     steepTeaBag
5     addLemon
6     pourInCup
7   end
8   def boilWater
9     puts 'boiling water'
10  end
11  def steepTeaBag
12    puts 'steeping tea'
13  end
14  def addLemon
15    puts 'adding lemon'
16  end
17  def pourInCup
18    puts 'Pouring in cup'
19  end
20 end

```

```

1 class Coffee
2   def prepareRecipe
3     boilWater
4     brewCoffeeGrinds
5     pourInCup
6     addSugarAndMilk
7   end
8   def boilWater
9     puts 'boiling water'
10  end
11  def brewCoffeeGrinds
12    puts 'dripping coffee through filter'
13  end
14  def pourInCup
15    puts 'Pouring in cup'
16  end
17  def addSugarAndMilk
18    puts 'adding sugar and milk'
19  end
20 end

```

- Otra clase tendria mucho codigo mas duplicado

Solucion: (Abstracta + Override en Clases separada)

```

1 class CaffeineBeberage
2   def prepareRecipe
3     boilWater
4     brew
5     pourInCup
6     addCondiments
7   end
8   def boilWater
9     puts 'boiling water'
10  end
11  def pourInCup
12    puts 'Pouring in cup'
13  end
14  def brew
15    raise NotImplementedError
16  end
17  def addCondiments
18    raise NotImplementedError
19  end
20 end

```

```

1 require_relative 'caffeine_beberage'
2 class Coffee < CaffeineBeberage
3   def brew
4     puts 'dripping coffee through filter'
5   end
6   def addCondiments
7     puts 'adding sugar and milk'
8   end
9 end

```

```

1 require_relative 'caffeine_beberage'
2 class Tea < CaffeineBeberage
3   def brew
4     puts 'steeping tea'
5   end
6   def addCondiments
7     puts 'adding lemon'
8   end
9 end

```

15. Patrones de Diseño (COMPORTAMIENTO)

- STRATEGY
- TEMPLATE METHOD
- OBSERVER

Patrones de Diseño

- Soluciones habituales a problemas frecuentes al diseñar Software
 - Es un concepto general para resolver un problema particular
-

1. Strategy

- Contexto + Estrategias
- Patrón de diseño de comportamiento que:
 - Permite definir una familia de ALGORITMOS,
 - Colocar cada uno de ellos en una CLASE SEPARADA
 - Hacer sus objetos intercambiables.

Problemas:

- En una app para viajeros con un mapa
 - V1: En el mapa se automatizan las rutas sobre la carretera
 - V2: Rutas a pie
 - V3: Rutas en Tren
 - V4: rutas en

La clase NAVEGADOR →

- Tiene mucha logica de fondo,

- hacer cambios se puede volver insostenible

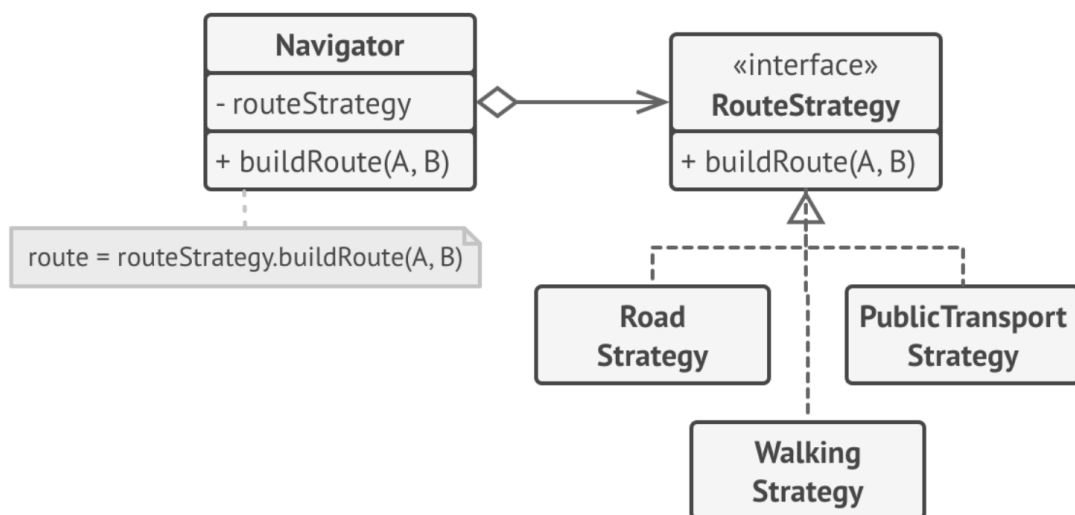
Solucion:

- Tomar la clase y separarla en distintas
- Extraer todos los algoritmos y colocarlos en CLASES SEPARADAS
- Cada clase será una **Estrategia**
- La clase original será el **Contexto**
- El cliente pasa la estrategia deseada al contexto
- El contexto llama a un METODO COMUN en las estrategias

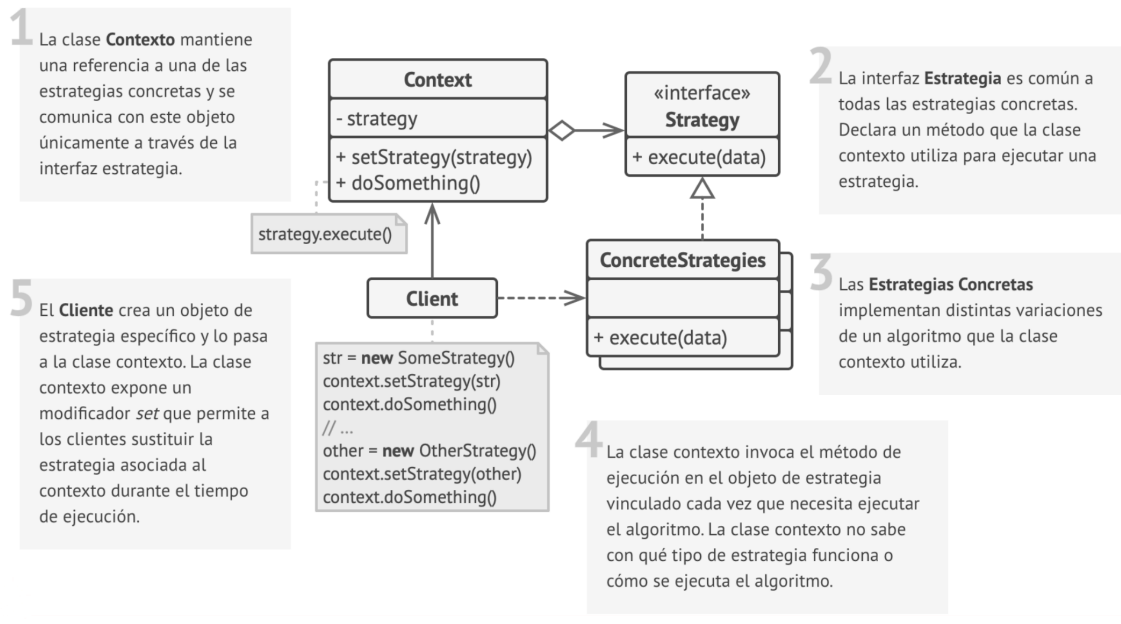
Ventajas

- Intercambiar algoritmos en tiempo de ejecucion
- Principio abierto/cerrado → Añadir nuevas estrategias sin modificar
- **Sustituye Herencia por Composición**

Diagrama



Estrategias de planificación de rutas.



Implementacion en codigo

```

class Context
# The Context maintains a reference to one of the Strategy objects
attr_writer :strategy

def initialize(strategy)
  @strategy = strategy
end

# provides a setter to change it at runtime.
def strategy=(strategy)
  @strategy = strategy
end

def do_some_business_logic
  # ...

  puts 'Context: Sorting data using the strategy (not sure how it\'ll do it)'
  result = @strategy.do_algorithm(%w[a b c d e])
  print result.join(',')

  # ...
  
```

```

    end
end

class Strategy
  # @abstract

  # @param [Array] data
  def do_algorithm(_data)
    raise NotImplementedError, "#{self.class} has not implemented method '
  end
end

# Concrete Strategies implement the algorithm while following the base Str
# interface. The interface makes them interchangeable in the Context.

class ConcreteStrategyA < Strategy
  # @param [Array] data
  #
  # @return [Array]
  def do_algorithm(data)
    data.sort
  end
end

class ConcreteStrategyB < Strategy
  # @param [Array] data
  #
  # @return [Array]
  def do_algorithm(data)
    data.sort.reverse
  end
end

# The client code picks a concrete strategy and passes it to the context
context = Context.new(ConcreteStrategyA.new)
puts 'Client: Strategy is set to normal sorting.'
context.do_some_business_logic

```

```
puts 'Client: Strategy is set to reverse sorting.'
context.strategy = ConcreteStrategyB.new
context.do_some_business_logic
```

OUTPUT

Client: Strategy is set to normal sorting.
Context: Sorting data using the strategy (not sure how it'll do it)
a,b,c,d,e

Client: Strategy is set to reverse sorting.
Context: Sorting data using the strategy (not sure how it'll do it)
e,d,c,b,a

Problema de Prueba:

```
class PaymentProcessor
  def initialize(user, amount, payment_type)
    @user = user
    @amount = amount
    @payment_type = payment_type
  end

  def process_payment
    case @payment_type
    when "credit_card"
      puts "Processing credit card payment for amount: #{@amount}"
      # Imagine more complex logic here
    when "paypal"
      puts "Processing PayPal payment for amount: #{@amount}"
      # Imagine more complex logic here
    when "bitcoin"
      puts "Processing Bitcoin payment for amount: #{@amount}"
      # Imagine more complex logic here
    else
      raise "Unsupported payment type: #{@payment_type}"
    end
  end
end

# Example of use
payment = PaymentProcessor.new("John Doe", 100.0, "credit_card")
payment.process_payment
```

- A. (3 pts) Explique el porque el código anterior no es abierto para extensiones ni cerrado para modificaciones.
- B. (7 pts) Modifique el código anterior utilizando el patrón strategy.

```

class PaymentProcessor
  def initialize(user, amount, payment_strategy)
    @user = user
    @amount = amount
    @payment_strategy = payment_strategy
  end

  def process_payment
    @payment_strategy.charge(@amount)
  end
end

class PaymentStrategy
  def charge(amount)
    raise NotImplementedError
  end
end

class CreditCard < PaymentStrategy
  def charge(amount)
    puts "Processing credit card payment for amount: $#{amount}"
  end
end

class PayPal < PaymentStrategy
  def charge(amount)
    puts "Processing PayPal payment for amount: $#{amount}"
  end
end

class BitCoin < PaymentStrategy
  def charge(amount)
    puts "Processing Bitcoin payment for amount: $#{amount}"
  end
end

# Example of use
payment = PaymentProcessor.new("John Doe", 100.0, CreditCard.new)
payment.process_payment

```

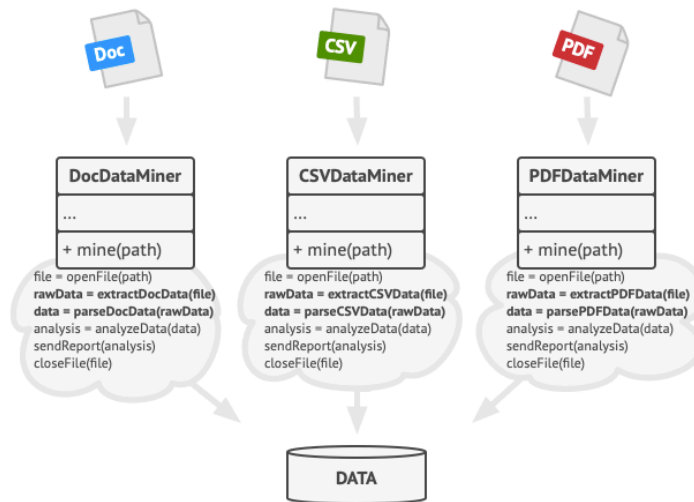
2. Template Method

- Patron que define el **esqueleto del algoritmo** en una **Superclase**
- Permite que las **Subclases sobrescriban** pasos del algoritmo (sin cambiar estructura)

Problemas:

- En una app para abrir documentos
 - V1 : Solo archivos DOC

- V2: Archivos CSV
- V3: Archivos PDF
- etc
- TENEMOS MUCHO CODIGO DUPLICADO



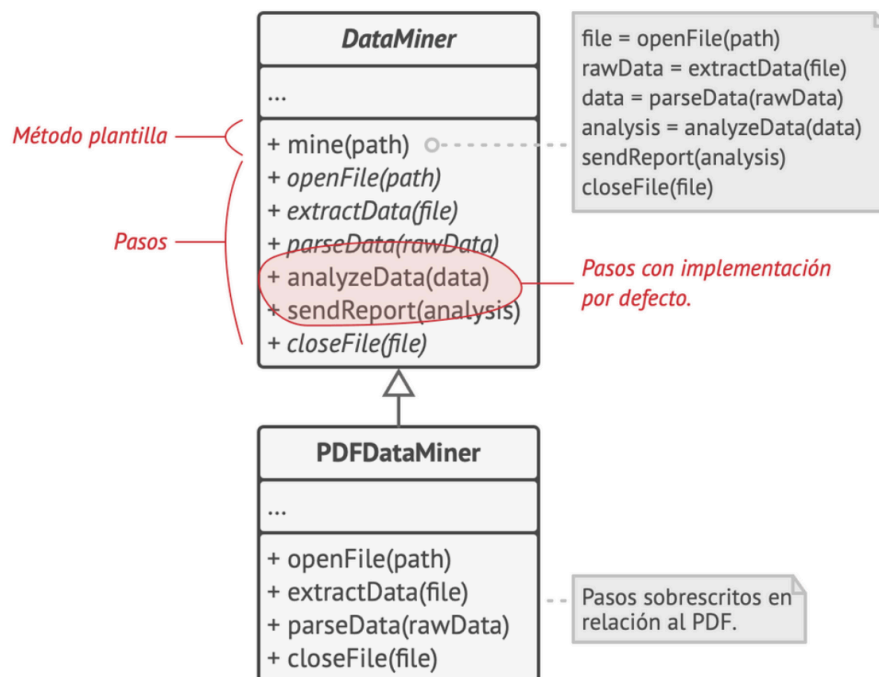
Las clases de minería de datos contenían mucho código duplicado.

Solución:

- Dividir un algoritmo en **varios pasos** → cada paso es un **método**
- La serie de llamadas a los metodos va en un **metodo Plantilla**
- Pasos pueden ser abstractos o tener implementacion

Cliente →

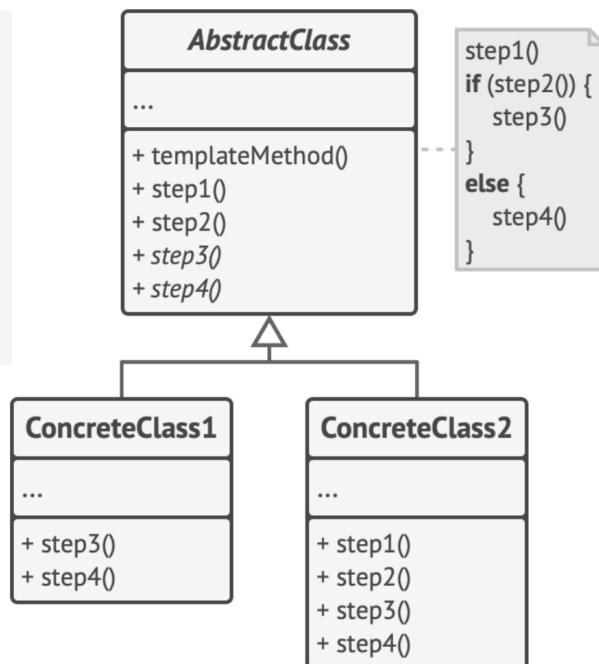
- Crea su Subclase,
- implementa pasos abstractos y sobrescribe los opcionales
- "NUNCA se Sobrescribe el metodo plantilla"



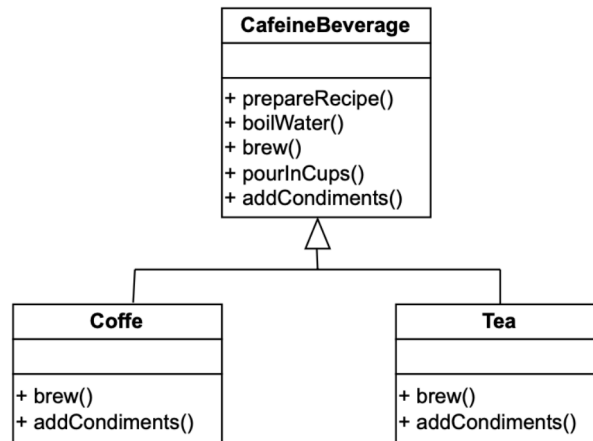
El método plantilla divide el algoritmo en pasos, permitiendo a las subclases sobrescribir estos pasos pero no el método en sí.

1 La **Clase Abstracta** declara métodos que actúan como pasos de un algoritmo, así como el propio método plantilla que invoca estos métodos en un orden específico. Los pasos pueden declararse abstractos o contar con una implementación por defecto.

2 Las **Clases Concretas** pueden sobrescribir todos los pasos, pero no el propio método plantilla.



Ejemplo Diagrama (ya visto)



EjemploCodigo

The Abstract Class defines a template method that contains a skeleton of some algorithm, composed of calls to (usually) abstract primitive operations.

```

class AbstractClass
# The template method defines the skeleton of an algorithm.
def template_method
  base_operation1
  required_operations1
  base_operation2
  hook1
  required_operations2
  base_operation3
  hook2
end

def base_operation1
  puts 'AbstractClass says: I am doing the bulk of the work'
end

def base_operation2
  puts 'AbstractClass says: But I let subclasses override some operations'
end

def base_operation3

```

```

    puts 'AbstractClass says: But I am doing the bulk of the work anyway'
  end

  # These operations have to be implemented in subclasses. → (Abstractas)
  def required_operations1
    raise NotImplementedError, "#{self.class} has not implemented method '#{_}'"
  end

  def required_operations2
    raise NotImplementedError, "#{self.class} has not implemented method '#{_}'"
  end

  # Subclasses may override them, but it's not mandatory
  def hook1; end
  def hook2; end
end

# -----
# Concrete classes have to implement all abstract operations of the base class
# They can also override some operations with a default implementation.
class ConcreteClass1 < AbstractClass
  def required_operations1
    puts 'ConcreteClass1 says: Implemented Operation1'
  end

  def required_operations2
    puts 'ConcreteClass1 says: Implemented Operation2'
  end
end

class ConcreteClass2 < AbstractClass
  def required_operations1
    puts 'ConcreteClass2 says: Implemented Operation1'
  end

  def required_operations2
    puts 'ConcreteClass2 says: Implemented Operation2'
  end
end

```

```

def hook1
  puts 'ConcreteClass2 says: Overridden Hook1'
end
end

#-----
# The client code calls the template method to execute the algorithm.
def client_code(abstract_class)
  # ...
  abstract_class.template_method
  # ...
end

puts 'Same client code can work with different subclasses:'
client_code(ConcreteClass1.new)
puts "\n"

puts 'Same client code can work with different subclasses:'
client_code(ConcreteClass2.new)

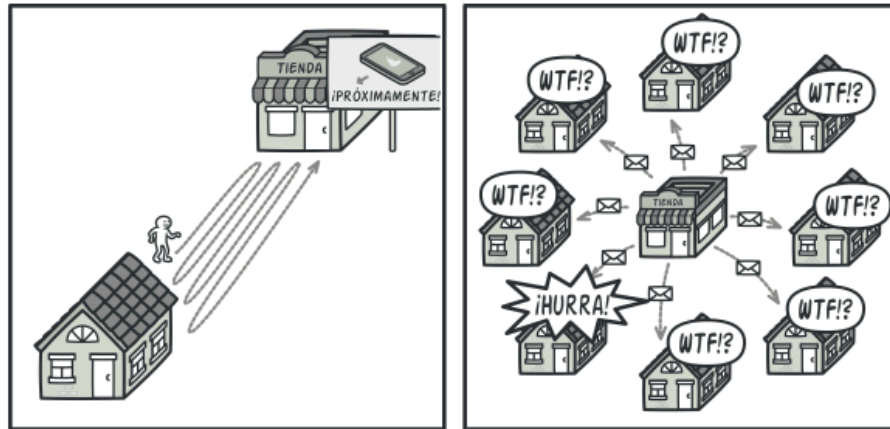
```

3. Observer 🧐

- Patron que permite definir un mecanismo de **Subscripción** para notificar a **varios objetos**
- Sobre un **Evento** sobre algun objeto

Problemas:

- Tienda con clientes, quieren saber cuando un producto llego
 - Cliente puede visitar la tienda todos los dias
 - La tienda podria enviar cientos de correos cada vez que haya un producto disponible

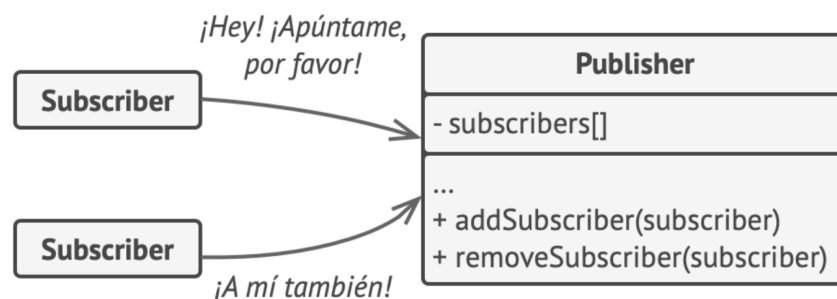


Visita a la tienda vs. envío de spam

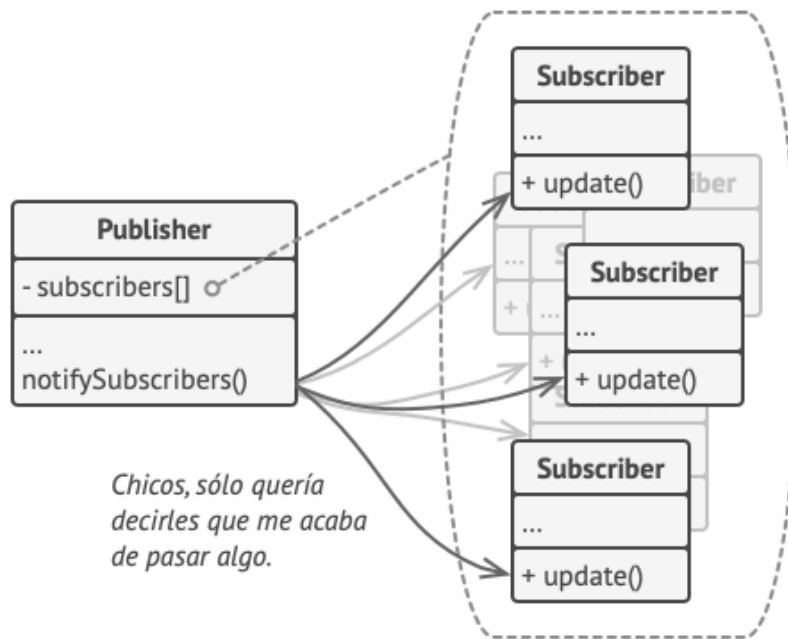
- **SUJETO:** El objeto el cual tiene el estado interesante para otros
- **NOTIFICADOR:** o publicador
- **SUSCRIPTORES:** Objetos que quieren conocer cambio de estado del notificador
- El patron sugiere: Añadir **Metodo de subscripción** a la clase notificadora

Ventajas:

- Principio abierto/cerrado → Introducir nuevas clases subscriptoras sin modificar código
- Se pueden establecer relaciones entre objetos durante el tiempo de ejecución



Un mecanismo de suscripción permite a los objetos individuales suscribirse a notificaciones de eventos.



El notificador notifica a los suscriptores invocando el método de notificación específico de sus objetos.

Codigo

```

# The Subject interface declares a set of methods for managing subscribers.
# @abstracta
class Subject
  # Attach an observer to the subject.
  def attach(observer)
    raise NotImplementedError
  end

  # Detach an observer from the subject.
  def detach(observer)
    raise NotImplementedError
  end

  # Notify all observers about an event.
  def notify
    raise NotImplementedError
  end
  
```

```

end
end

# -----

class ConcreteSubject < Subject
  # For the sake of simplicity, the Subject's state, essential to all
  # subscribers, is stored in this variable.
  attr_accessor :state

  def initialize
    @observers = []
  end

  # @param [Observer] observer
  def attach(observer)
    puts 'Subject: Attached an observer.'
    @observers << observer
  end

  # @param [Observer] observer
  def detach(observer)
    @observers.delete(observer)
  end

  # ----- The subscription management methods -----

  # Trigger an update in each subscriber.
  def notify
    puts 'Subject: Notifying observers...'
    @observers.each { |observer| observer.update(self) }
  end

  def some_business_logic
    puts "\nSubject: I'm doing something important."
    @state = rand(0..10)

    puts "Subject: My state has just changed to: #{@state}"
  end
end

```

```

    notify
  end
end

# ----- Observadores -----
class Observer
  # Receive update from subject.
  def update(_subject)
    raise NotImplementedError
  end
end

class ConcreteObserverA < Observer

  def update(subject)
    puts 'ConcreteObserverA: Reacted to the event' if subject.state < 3
  end
end

class ConcreteObserverB < Observer

  def update(subject)
    return unless subject.state.zero? || subject.state >= 2
    puts 'ConcreteObserverB: Reacted to the event'
  end
end

#----- The client code -----
subject = ConcreteSubject.new

observer_a = ConcreteObserverA.new
subject.attach(observer_a)

observer_b = ConcreteObserverB.new
subject.attach(observer_b)

subject.some_business_logic
subject.some_business_logic

```

```
subject.detach(observer_a)
```

```
subject.some_business_logic
```

Ejercicio de Prueba (Metodo Observador + Diagrama Secuencia)

Pregunta 1

Se ha construido un trozo de código que utiliza un patrón observador para vigilar y actuar frente a cambios en el sujeto que en este caso es un objeto de la clase Employee. Los objetos de esta clase representan empleados y tienen solo 3 atributos: *name*, *job* y *salary*.

Los objetos observadores corresponden a instancias de 2 clases: JobsLog y SalariesLog y, como buenos observadores se activan cuando el sujeto sufre cambios. Para simplificar, supondremos que la reacción de cada uno de ellos ante cambios en el estado del sujeto es muy simple:

Un objeto JobsLog imprimirá el nuevo cargo del empleado de la siguiente forma:

*** Pepe es un analista *** (suponiendo que es Pepe quien cambió)

Un objeto SalariesLog imprimirá el nuevo sueldo del empleado de la siguiente forma:

*** el sueldo de Pepe es 1.000.000 *** (suponiendo que es Pepe quien cambió)

A continuación un trozo de código que prueba el funcionamiento:

```
jaime = Employee.new('Jaime', 'Programador Junior', 1000000)
jobslog = JobsLog.new
salarieslog = SalariesLog.new
jaime.add_observer( jobslog )
jaime.add_observer( salarieslog )
jaime.job = "Programador"
jaime.salary = 1500000
jaime.delete_observer( jobslog )
jaime.job = "Programador Senior"
jaime.salary = 2000000
```

El output que produce es el siguiente:

```
*** Jaime es un Programador ***
*** el sueldo de Jaime es 1000000 ***
*** Jaime es un Programador ***
*** el sueldo de Jaime es 1500000 ***
*** el sueldo de Jaime es 1500000 ***
*** el sueldo de Jaime es 2000000 ***
```

a) Escriba las clases Employee, SalariesLog y JobsLog y dibuje el diagrama de clases correspondiente

b) Dibuje un diagrama de secuencia que ilustre toda la secuencia de prueba y que permita explicar el output generado

Solución

```
class Employee
  attr_reader :name, :job
  attr_reader :salary
  def initialize( name, title, salary )
    @name = name
    @job = job
    @salary = salary
    @observers = []
  end

  def salary=(new_salary)
    @salary = new_salary
    notify_observers
  end

  def job=(new_job)
    @job = new_job
    notify_observers
  end

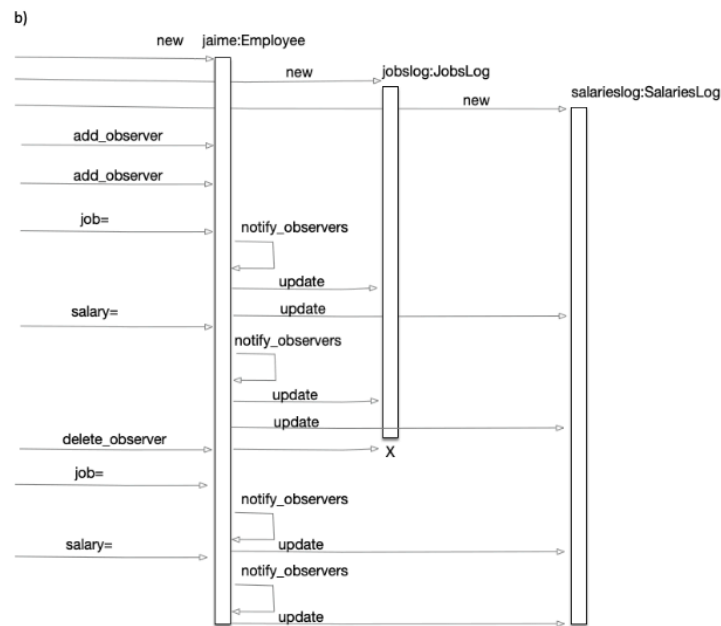
  def notify_observers
    @observers.each do |observer| observer.update(self)
  end

  def add_observer(observer)
    @observers << observer
  end

  def delete_observer(observer)
    @observers.delete(observer)
  end
end

class JobsLog
  def update( changed_employee )
    puts("*** #{changed_employee.name} es un #{changed_employee.job} ***")
  end
end

class SalariesLog
  def update( changed_employee )
    puts("*** el sueldo de #{changed_employee.name} es #{changed_employee.salary} ***")
  end
end
```



Observar que hay 6 veces que se invoca un update: dos para cada uno de los dos observadores primero:

```

*** Jaime es un Programador ***
*** el sueldo de Jaime es 1000000 ***
*** Jaime es un Programador ***
*** el sueldo de Jaime es 1500000 ***
  
```

y luego dos para el observador de sueldos que queda después de eliminar el de jobs.

```

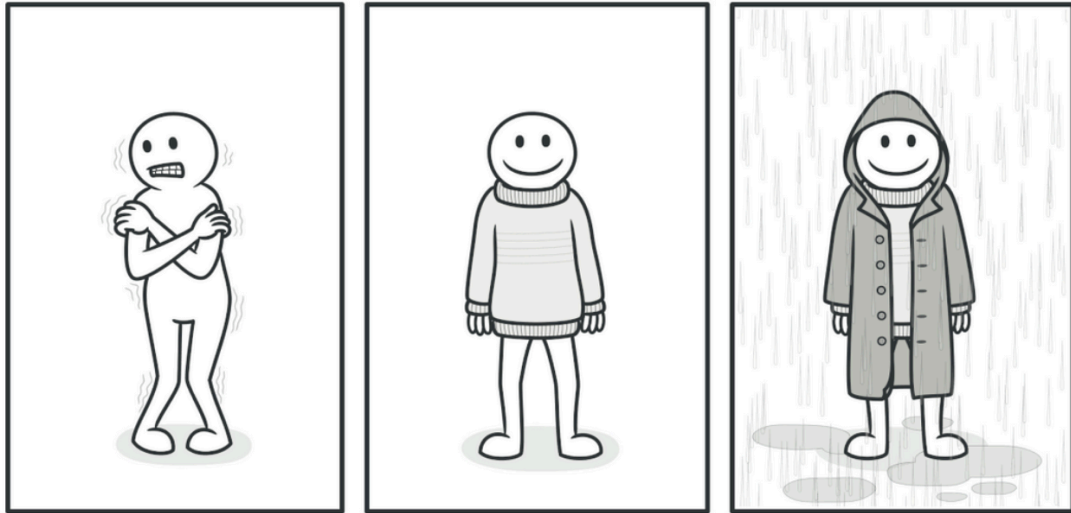
*** el sueldo de Jaime es 1500000 ***
*** el sueldo de Jaime es 2000000 ***
  
```

16. Patrones de Diseño (ESTRUCTURALES)

- DECORATOR
- COMPOSITE
- ADAPTER
- PROXY

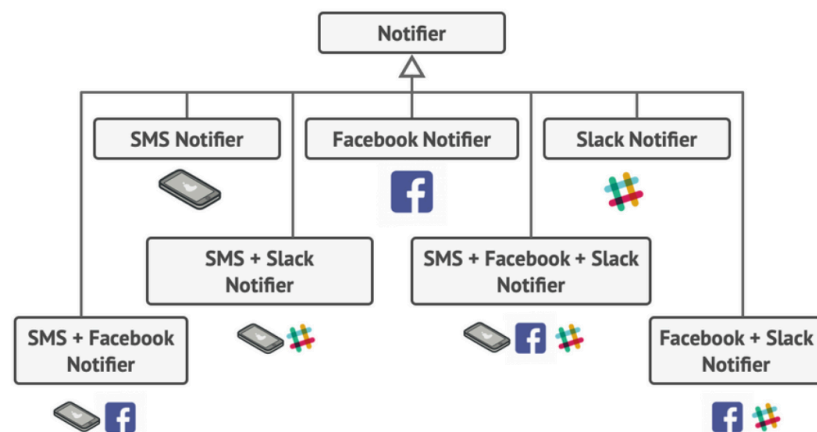
4. DECORATOR 🍕

- Permite **AÑADIR FUNCIONALIDADES** a Objetos
- Colocando estos **dentro de otros objetos encapsuladores** que contienen a estas funcionalidades



Problemática

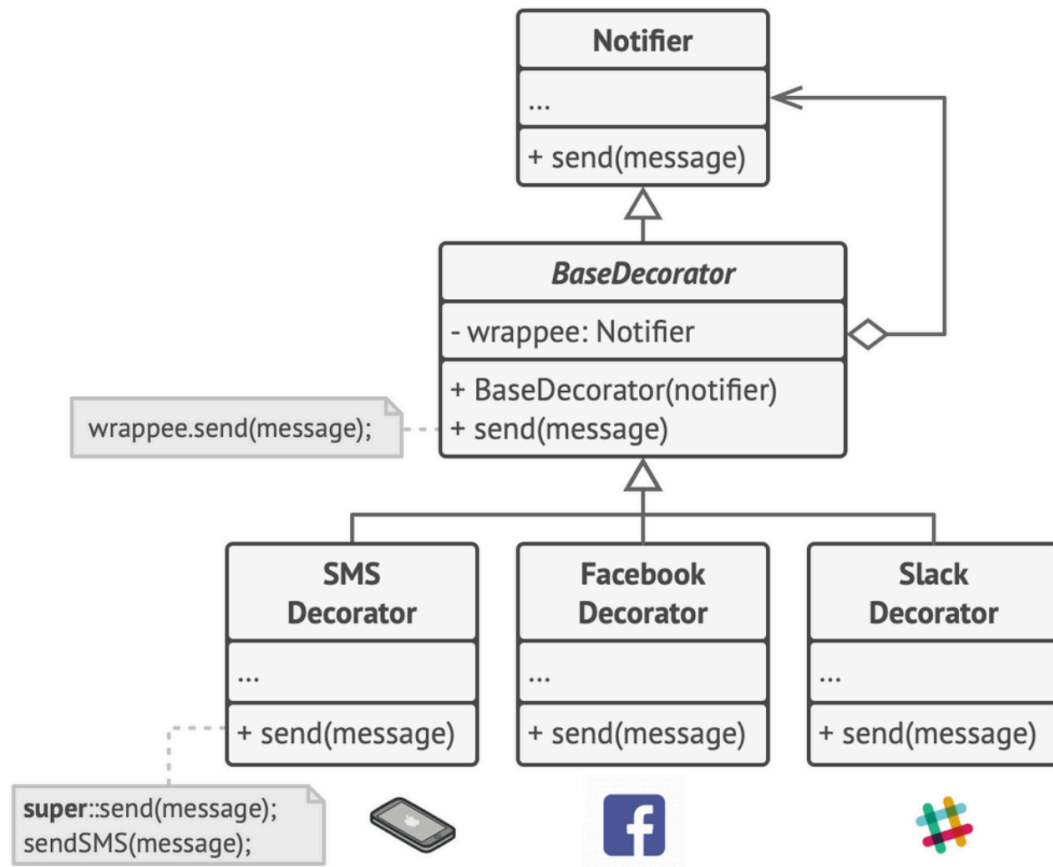
- App de Notificaciones, solo se envían por un canal
- ¿Que pasa si quiero recibir en varios canales?



Explosión combinatoria de subclases.

Solución

- Añadir **Composicion**: Un objeto tiene referencia a otro y le delega trabajo
- **Objeto Decorador** tiene como **atributo al Objeto Decorado**
- Cuando un **metodo del objeto** Decorado es llamado, el **Decorador llama** al metodo del **decorado** y luego realiza acciones adicionales



Varios métodos de notificación se convierten en decoradores.

⚖️ Pros y contras

- ✓ Puedes extender el comportamiento de un objeto sin crear una nueva subclase.
- ✓ Puedes añadir o eliminar responsabilidades de un objeto durante el tiempo de ejecución.
- ✓ Puedes combinar varios comportamientos envolviendo un objeto con varios decoradores.
- ✓ *Principio de responsabilidad única.* Puedes dividir una clase monolítica que implementa muchas variantes posibles de comportamiento, en varias clases más pequeñas.
- ✗ Resulta difícil eliminar un wrapper específico de la pila de wrappers.
- ✗ Es difícil implementar un decorador de tal forma que su comportamiento no dependa del orden en la pila de decoradores.
- ✗ El código de configuración inicial de las capas pueden tener un aspecto desagradable.

Ejemplo

- Supongamos que quiero vender una piza y tenemos el código:

```
class Pizza
  def print
    raise NotImplementedError
  end

  def cost
    raise NotImplementedError
  end
end
```

```
require_relative 'pizza'
class PineableHamCheesePizza < Pizza
  def print
    puts 'normal bred'
    puts 'cheese'
    puts 'ham'
    puts 'pineapple'
  end

  def cost
    (20 + 5 + 5) * 2
    #cada porcion de pineable duplica el costo
  end
end
```

- Actualmente se agregan los toppings de manera estática. Si queremos hacer una pizza de doble queso hay que crear otra clase.
- Tenemos código duplicado entre distintos tipos de pizza.

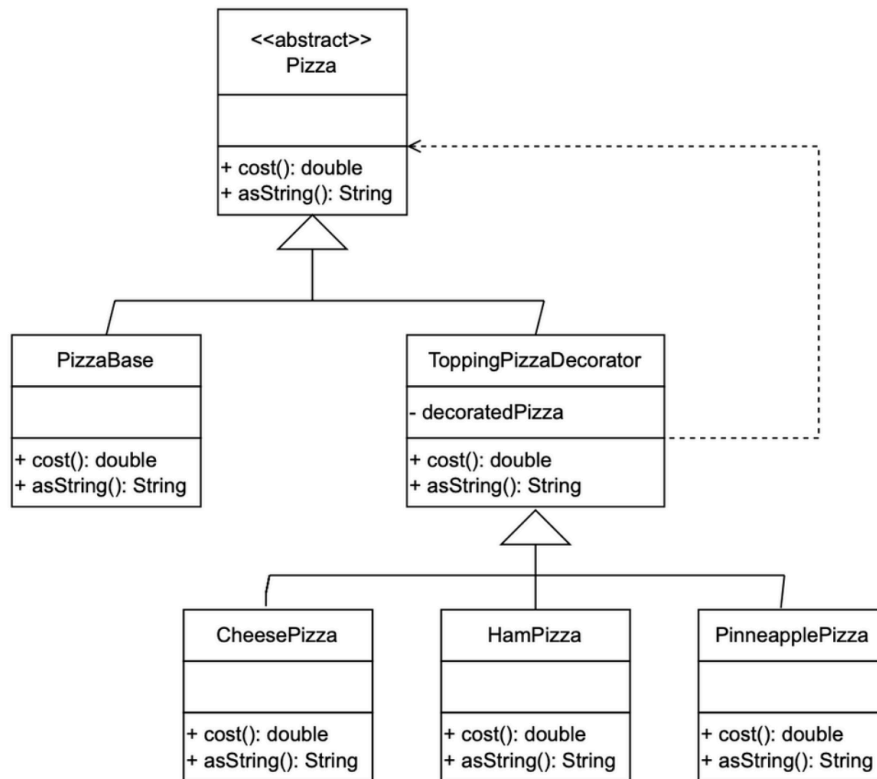
```
require_relative 'ham_cheese_normal_pizza'
require_relative 'pineapple_ham_cheese_normal_pizza'

hawaii_pizza = PineableHamCheesePizza.new
hawaii_pizza.print
puts "#{hawaii_pizza.cost}"

basic_pizza = HamCheesePizza.new
basic_pizza.print
puts "#{basic_pizza.cost}"
```

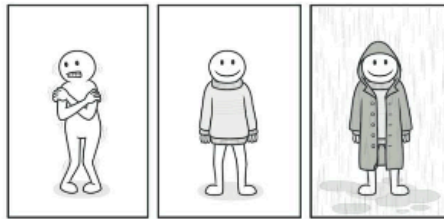
Solucion:

- Herencia y Dependencia



Ejercicio Prueba

Pregunta 5. En este ejercicio usted debe implementar el patrón Decorator que permita modelar una persona que puede tener diferentes tipos de abrigos que se puede poner uno encima de otro. Por ejemplo, considere la siguiente imagen:



Obtienes un efecto combinado vistiendo varias prendas de ropa.

En la primera imagen, se ve una persona sin ningún abrigo, en la segunda imagen, una persona con un suéter, y en la última imagen una persona con un suéter y una parca. El modelo de clases debe permitir instanciar personas con diferentes combinaciones de abrigos.

- **(5 pts)** Implemente el patrón Decorator que permita modelar personas con diferentes tipos de abrigos. Debe escribir el código correspondiente a cada clase.
- **(5 pts)** Agregue una operación llamada `temperatura_corporal`, que devuelva la temperatura esperada del cuerpo de la persona que tiene N abrigos.
 - Una persona tiene una temperatura corporal de 37 grados.
 - Un suéter se espera que aumente la temperatura de la persona 2 grados.
 - Una parca se espera que aumente la temperatura de la persona en 5 grados.
 - Por ejemplo, una persona con dos suéteres y una parca debe tener una temperatura de 47 grados ($37+2+2+5$).

```
class PersonInterface
  def temperatura_corporal
    raise NotImplementedError
  end
end

class Person < PersonInterface
  def temperatura_corporal
    37
  end
end

# Aqui Estamos entregando el Objeto Decorado al Decorador
class PersonDecorator < PersonInterface
  def initialize(person)
    @person = person
  end

  def temperatura_corporal
```

```

    @person.temperatura_corporal
    end
end

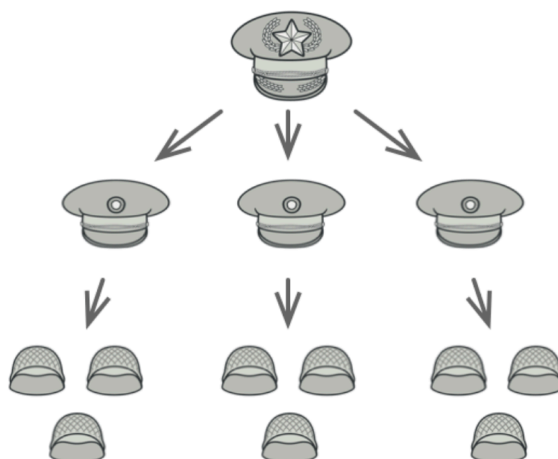
class Sweater < PersonDecorator
  def temperatura_corporal
    super + 2
  end
end

class Coat < PersonDecorator
  def temperatura_corporal
    super + 5
  end
end

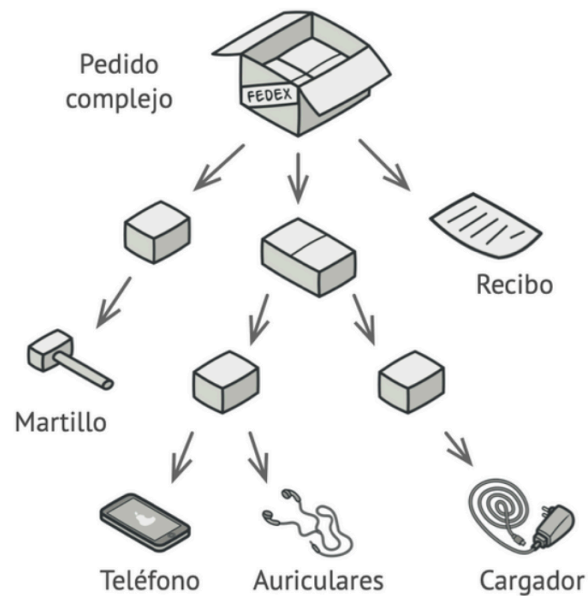
```

5. COMPOSITE

- Patrón de diseño estructural que te permite componer objetos en [estructuras de árbol](#)
- Trabaja con esas estructuras como si fueran objetos individuales.

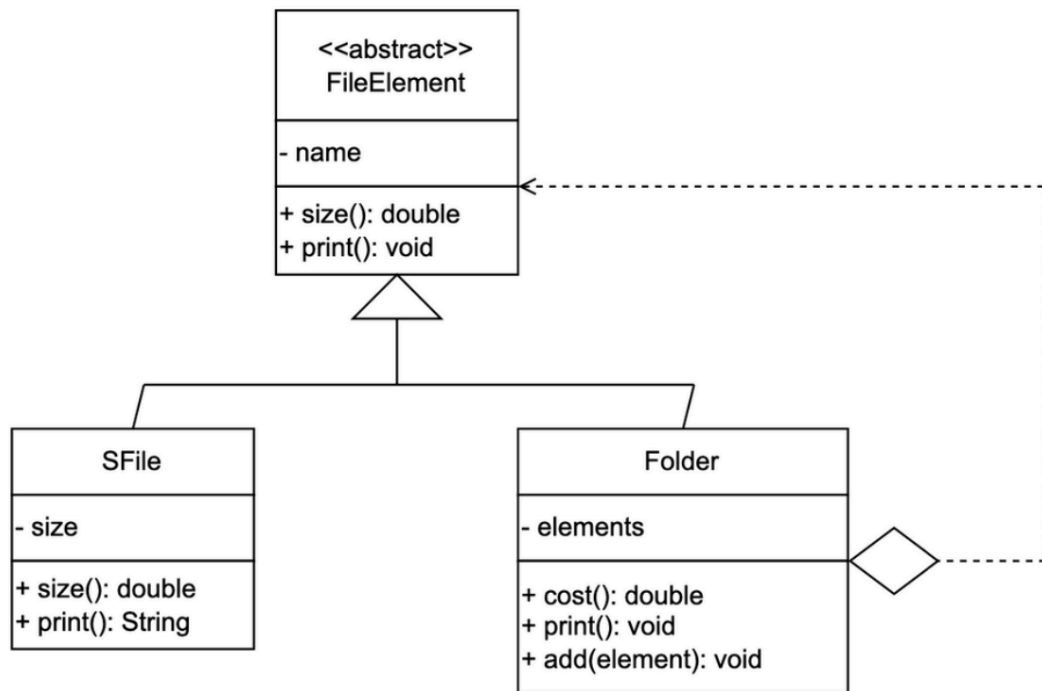


- Este patron solo tiene sentido si tenemos una estrucuta de arbol
- Por ejemplo: Paquete tiene cajas y objetos y una caja puede tener otra caja hasta llegar a un producto



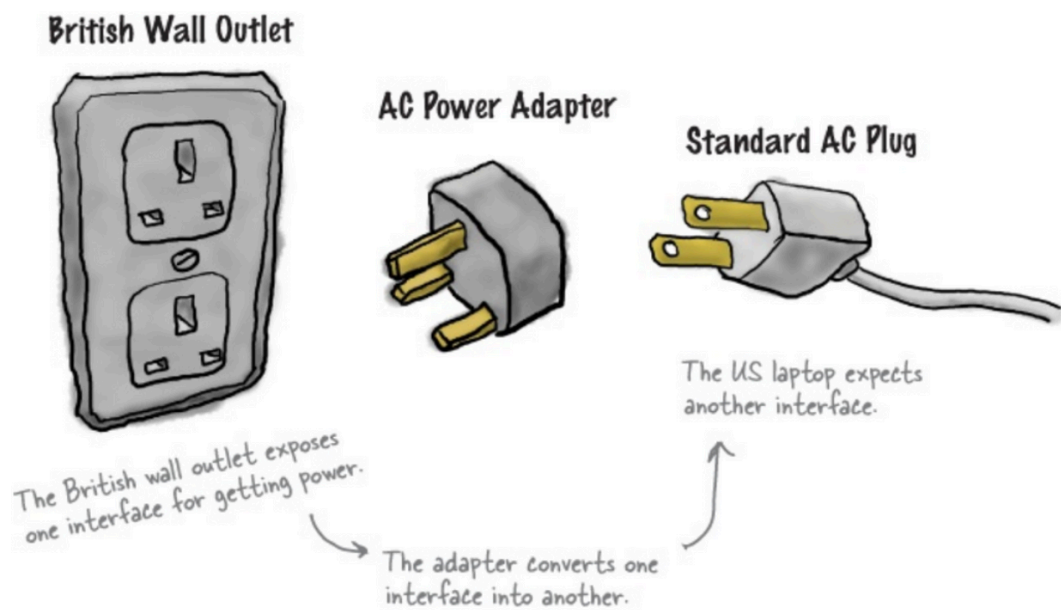
Forma de Actuar: Recursiva

1. Para el producto devuelve el precio
2. Para una caja, recorre cada articulo dentro y devuelve un total por caja
3. Si uno de estos articulos es una caja, esa caja tambien repasa su contenido y asi sucesivamente



6. Adapter

- Permite la colaboracion entre Objetos con Interfaces Incompatibles

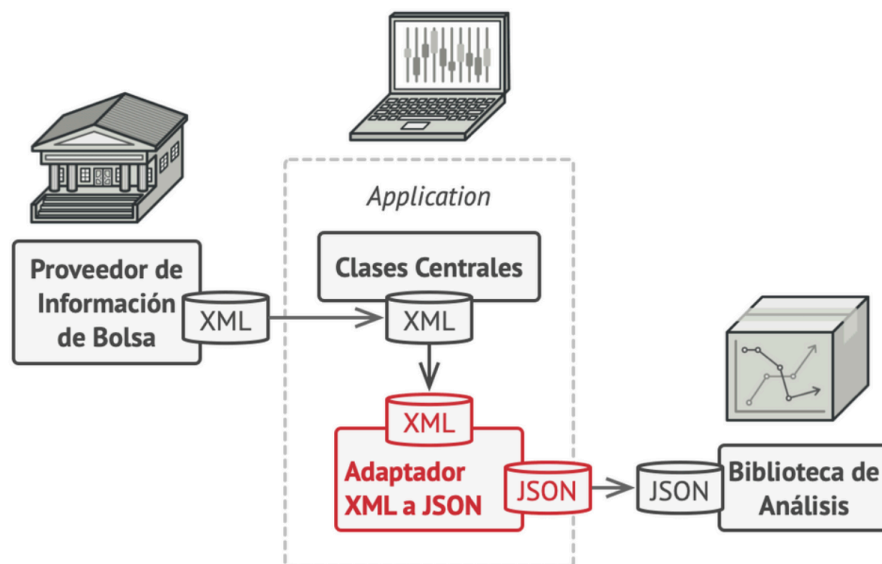


Problemas

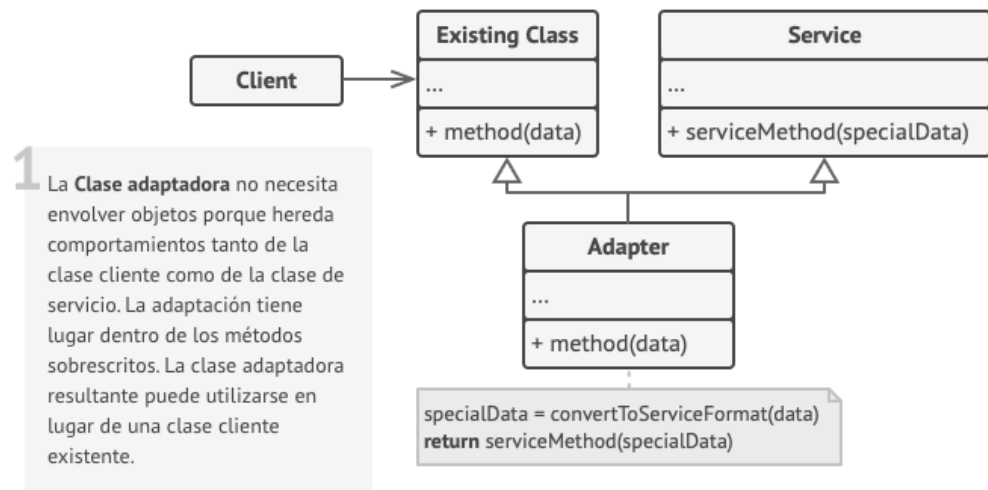
- Incompatibilidad de formatos
- Ej: App de monitoreo que descarga info en XML
 - V2: Integración con biblioteca que solo funciona con JSON

Solución

- Crear un adaptador para convertir la interfaz de un objeto → Así otro podra entenderla



1. El adaptador obtiene una interfaz compatible con uno de los objetos existentes
2. El objeto existente puede invocar con la interfaz los métodos del adaptador.
3. Al recibir una llamada, el adaptador pasa la solicitud al segundo objeto, pero en el formato que este objeto espera.



Problema Ejemplo:

```

class ConsoleClient
  def initialize(service)
    @service = service
  end
  def run
    rut = "22.456.456-2"
    clave = "secreta"
    if @service.login(rut,clave) then
      puts "autenticacion exitosa"
    else
      puts "autenticacion fallida"
    end
  end
end

class LoginService
  def initialize()
    @users = { "22456456-2" => "secreta",
               "11789789-3" => "password",
               "33456123-4" => "clave" }
  end
  def login_user(rut, pass)
    return @users[rut.strip] == pass.strip
  end
end

service = LoginService.new
client = ConsoleClient.new(service)
client.run
  
```

Solución Ejemplo:

Motivación - Solución

```

class ConsoleClient
  def initialize(service)
    @service = service
  end
  def run
    rut = "22.456.456-2"
    clave = "secreta"
    if @service.login(rut,clave) then
      puts "autenticacion exitosa"
    else
      puts "autenticacion fallida"
    end
  end
end

class LoginServiceAdapter
  def initialize(originalService)
    @originalService = originalService
  end
  def login(rut, pass)
    adaptedrut = rut.gsub(".", "")
    return @originalService.login_user(adaptedrut, pass)
  end
end

service = LoginService.new
adaptador = LoginServiceAdapter.new(service)
client = ConsoleClient.new(adaptador)
client.run
  
```

```

# La clase Objetivo define la interfaz usada por el cliente.
class Objetivo
  def solicitud
    'Objetivo: El comportamiento por defecto del objetivo.'
  end
end

# La clase Adaptado contiene una funcionalidad útil, pero su interfaz es
class Adaptado
  def solicitud_especifica
    '.eetpadA eht fo roivaheb laicepS'
  end
end

# El Adaptador hace que la interfaz del Adaptado sea compatible con la
class Adaptador < Objetivo
  def initialize(adaptado)
    @adaptado = adaptado
  end

  def solicitud
    "Adaptador: (TRADUCIDO) #{@adaptado.solicitud_especifica.reverse}"
  end
end

# El código del cliente soporta todas las clases que siguen la interfaz de
def codigo_cliente(objetivo)
  puts objetivo.solicitud
end

puts 'Cliente: Puedo trabajar bien con los objetos del Objetivo:'
objetivo = Objetivo.new
codigo_cliente(objetivo)

adaptado = Adaptado.new
puts 'Cliente: La clase Adaptado tiene una interfaz extraña. Mira, no la

```

```
puts "Adaptado: #{adaptado.solicitud_especifica}"
```

```
puts 'Cliente: Pero puedo trabajar con ella a través del Adaptador:'  
adaptador = Adaptador.new(adaptado)  
codigo_cliente(adaptador)
```

7. PROXY

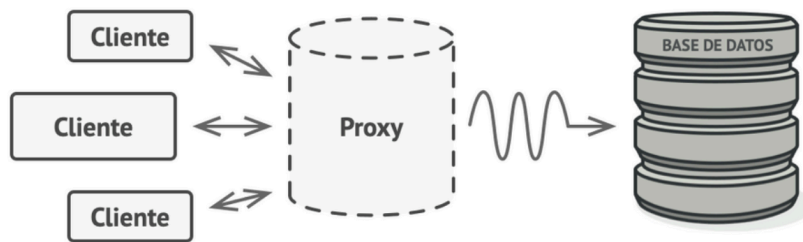
- Proporciona un Sustituto o marcador de posición para otro Objeto
- Controla el acceso al objeto original
- Permite hacer algo antes o después de que la solicitud llegue al objeto principal

Problema:

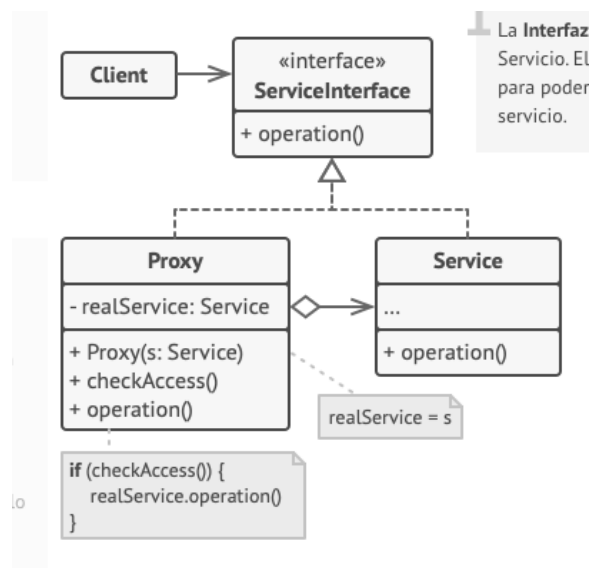
- Un objeto enorme que consume muchos recursos, se utiliza a veces y no siempre
- Podemos crear el objeto solo cuando sea necesario → genera mucho código

Solucion:

- Crear la clase "Proxy" con la misma interfaz del objeto original
- Se actualiza la app para pasars el proxy a todos los clientes del objeto original
- Si necesitamos ejecutar algo entremedio, no necesitamos cambiar la Clase



El proxy se camufla como objeto de la base de datos. Puede gestionar la inicialización diferida y el caché de resultados sin que el cliente o el objeto real de la base de datos lo sepan.



Problema:

Motivación - Problema

```
class ConsoleClient
  def initialize(service)
    @service = service
  end
  def run
    rut = "22456456-2"
    clave = "secreta"
    if @service.login(rut, clave) then
      puts "autenticación exitosa"
    else
      puts "autenticación fallida"
    end
  end
end
```

```
class LoginService
  def initialize()
    @users = { "22456456-2" => "secreta",
               "11789789-3" => "password",
               "33456123-4" => "clave" }
  end
  def login_user(rut, pass)
    return @users[rut.strip] == pass.strip
  end
end

service = LoginService.new
client = ConsoleClient.new(service)
client.run
```

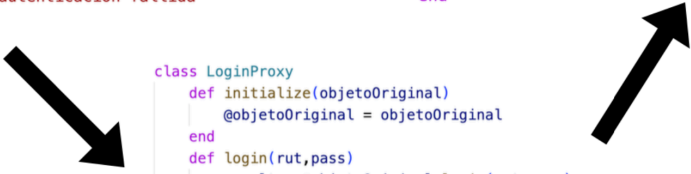
¿Cómo añadir un mensaje en consola (Log) cada vez que un cliente hace un login fallido?
Requisito es no tocar ninguna de las dos clases.

Solución:

```
class ConsoleClient
  def initialize(service)
    @service = service
  end
  def run
    rut = "22456456-2"
    clave = "secreta"
    if @service.login(rut,clave) then
      puts "autenticacion exitosa"
    else
      puts "autenticacion fallida"
    end
  end
end

class LoginService
  def initialize()
    @users = { "22456456-2" => "secreta2",
               "11789789-3" => "password",
               "33456123-4" => "clave" }
  end
  def login(rut, pass)
    return @users[rut.strip] == pass.strip
  end
end

class LoginProxy
  def initialize(objetoOriginal)
    @objetoOriginal = objetoOriginal
  end
  def login(rut,pass)
    result = @objetoOriginal.login(rut,pass)
    if result == false then
      puts "failed login attempt #{rut}"
    end
    return result
  end
end
```



```
class Sujeto
  # @abstract
  def solicitud
    raise NotImplementedError
  end
end
```

```
class SujetoReal < Sujeto
  def solicitud
    puts 'SujetoReal: Manejando solicitud.'
  end
end
```

El Proxy tiene una interfaz idéntica al SujetoReal.

```
class Proxy < Sujeto
  # @param [SujetoReal] sujeto_real
  def initialize(sujeto_real)
    @sujeto_real = sujeto_real
  end
```

Las aplicaciones más comunes del patrón Proxy son la carga diferida:

```

# el control de acceso, el registro, etc.
# Un Proxy puede realizar una de estas
# cosas y luego, dependiendo del resultado, pasar la ejecución al mismo
# método en un objeto SujetoReal vinculado.
def solicitud
  return unless verificar_acceso

  @sujeto_real.solicitud
  registrar_acceso
end

def verificar_acceso
  puts 'Proxy: Verificando acceso antes de realizar una solicitud real.'
  true
end

def registrar_acceso
  print 'Proxy: Registrando la hora de la solicitud.'
end
end

# El código del cliente se supone que debe funcionar con todos los objetos
# proxies) a través de la interfaz Sujeto para soportar tanto sujetos reales
# proxies. En la vida real, sin embargo, los clientes trabajan mayormente
# directamente. En este caso, para implementar el patrón más fácilmente
# tu proxy de la clase del sujeto real.
def codigo_cliente(sujeto)
  # ...

  sujeto.solicitud

  # ...
end

puts 'Cliente: Ejecutando el código del cliente con un sujeto real:'
sujeto_real = SujetoReal.new
codigo_cliente(sujeto_real)

```

```
puts "\n"
```

```
puts 'Cliente: Ejecutando el mismo código del cliente con un proxy:'  
proxy = Proxy.new(sujeto_real)  
codigo_cliente(proxy)
```

EJEMPLO DE PRUEBA (ADAPTER + PROXY)

Pregunta 4. Considere el siguiente segmento de código que fue extraído de la aplicación WeChat (la versión china de Whatsapp).

```
class Contact  
  ...  
  def receiveMessage(msg)  
    puts "you receive this message #{msg}"  
  ...  
  end  
  ...  
end  
class MessageSender  
  ...  
  def initialize(cnt)  
    @contact = cnt  
  ...  
  end  
  def sendMessage(msg)  
    @contact.receiveMessage(msg)  
  ...  
  end  
  ...  
end  
secretContact = Contact.new  
sender = MessageSender.new(secretContact)  
sender.sendMessage('saludos desde marte')
```

- **(5 pts)** Implementar el **patrón proxy** de forma tal que el objeto proxy imprima en consola "Potencial Capitalista" cuando un mensaje enviado tenga la palabra *Trump*.
- **(5 pts)** Implementar el **patrón adapter** para que cuando alguien envíe la palabra "F", adapte el mensaje enviando el texto "@#@\$" en su lugar.

Solucion:

Pregunta 4

Solución. Para cada inciso escribir la clase Proxy y Adapter era suficiente.

Proxy.

```
class ProxyContact
  def initialize(originalContact)
    @originalContact = originalContact
  end

  def receiveMessage(msg)
    if msg.include? "Trump"
      puts "Potencial Capitalista"
    end
    @originalContact.receiveMessage(msg)
  end
end
```

La parte verde puede variar (puede ser un igual por ejemplo), la idea es que el estudiante escriba un código que dé a entender que busca la palabra Trump en el mensaje. Es posible que haya usado otro método o función distinta. **No importa si el estudiante no recordó el nombre exacto de la función.**

Adapter.

```
class AdapterContact
  def initialize(originalContact)
    @originalContact = originalContact
  end

  def receiveMessage(msg)
    newMsg = msg.replace("F", "@#@$")
    @originalContact.receiveMessage(newMsg)
  end
end
```

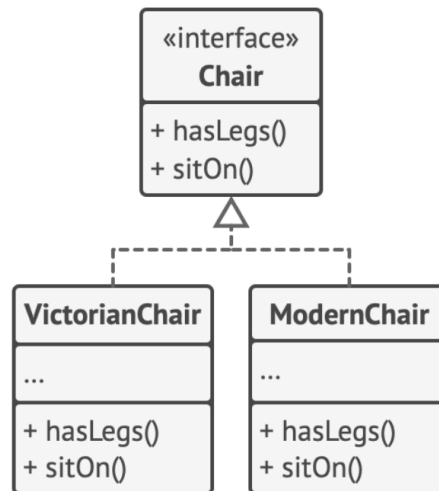
La parte verde puede variar, la idea es que el estudiante escriba un código que dé a entender que quiere reemplazar la palabra F por "@#@\$". Es posible que haya usado otro método o función distinta. **No importa si el estudiante no recordó el nombre exacto de la función. También es válido si el estudiante busca mensajes iguales a "F" (msg=="F").**

17. Patrones Diseño (CREACIONALES)

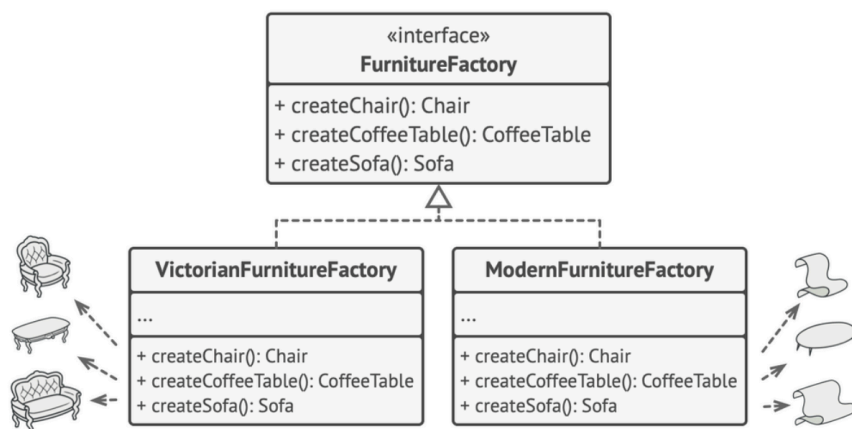
- ABSTRACT FACTORY
- BUILDER
- FACTORY
- SINGLETON

8. ABSTRACT FACTORY

- Nos permite producir familias de Objetos relacionados sin especificar clases concretas
- Ej: Familia de productos: Silla + Sofa + Mesa
- *REQUISITO: No cambiar código existente al añadir productos o familias*



- Mecanismo: → Crear una interfaz para cada producto diferente de la familia de productos
- Todas las variantes del producto seguirán esa interfaz
- Declaramos la **FABRICA ABSTRACTA**: interfaz de métodos de creación para todos los productos de una familia
- Los métodos devuelven productos abstractos



Cada fábrica concreta se corresponde con una variante específica del producto.

1. Crear Fabrica Abstracta

```
class FabricaAbstracta
  def crear_producto_a
    raise NotImplementedError
  end

  def crear_producto_b
    raise NotImplementedError
  end
end
```

2. Crear Fabrica Concreta

```
class FabricaConcreta < FabricaAbstracta
  def crear_producto_a
    ProductoConcretoA.new
  end

  def crear_producto_b
    ProductoConcretoB.new
  end
end
```

3. Definir Productos Abstractos

```
class ProductoAbstractoA
  def funcion_util_a
    raise NotImplementedError
  end
end

class ProductoAbstractoB
  def funcion_util_b
    raise NotImplementedError
  end
end
```

```

    def otra_funcion_util_b(_colaborador)
      raise NotImplementedError
    end
  end
end

```

4. Definir Productos Concretos

```

class ProductoConcretoA < ProductoAbstractoA
  def funcion_util_a
    'Resultado del producto A.'
  end
end

```

```

class ProductoConcretoB < ProductoAbstractoB
  def funcion_util_b
    'Resultado del producto B.'
  end

```

```

  def otra_funcion_util_b(colaborador)
    resultado = colaborador.funcion_util_a
    "Resultado del B colaborando con el (#{resultado})"
  end
end

```

5. Código del Cliente → Usa fabricas abstractas para crear productos

```

def codigo_cliente(fabrica)
  producto_a = fabrica.crear_producto_a
  producto_b = fabrica.crear_producto_b

  puts producto_b.funcion_util_b
  puts producto_b.otra_funcion_util_b(producto_a)
end

```

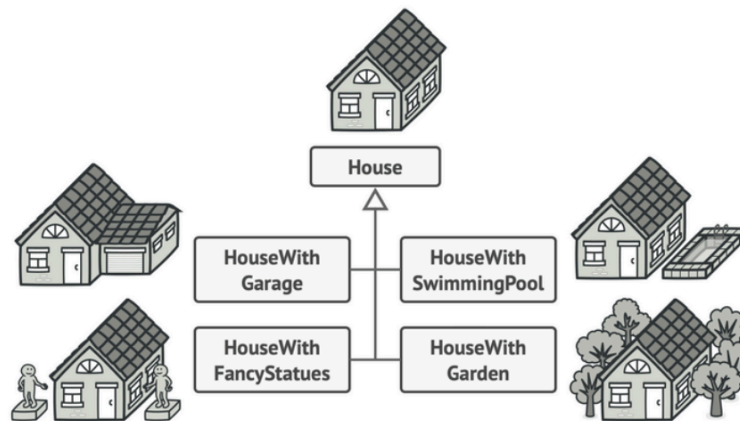
```

# Ejecutar código del cliente con una fabrica concreta
puts 'Cliente: Probando el código del cliente con una fábrica concreta:'
codigo_cliente(FabricaConcreta.new)

```

9. BUILDER

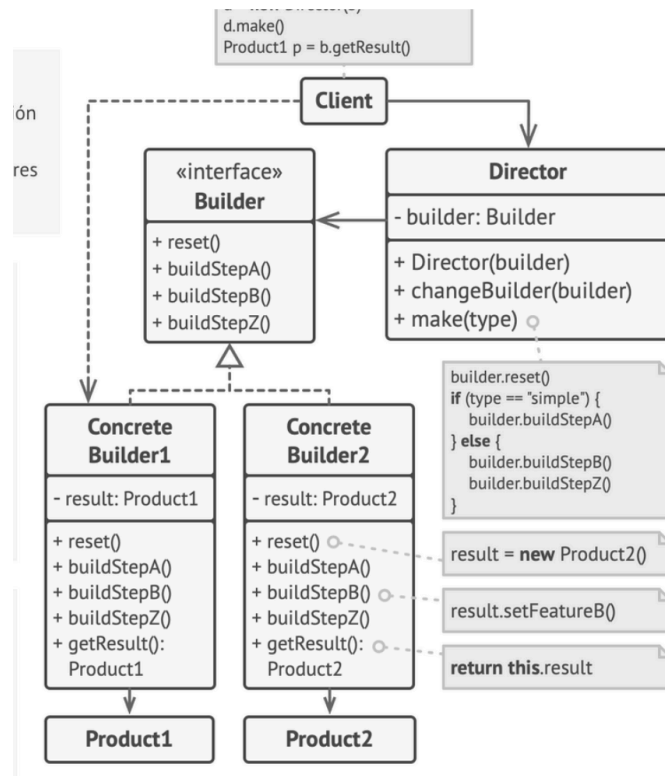
- Nos permite construir objetos complejos Paso a Paso
- Nos permite producir distintos tipos y representaciones de un objeto
- Ej: Queremos diferentes tipos de casas con atributos distintos → Muchas Subclases!



Crear una subclase por cada configuración posible de un objeto puede complicar demasiado el programa.

Solucion:

1. Sacar el codigo de construccion del objeto de su propia Clase
2. Colocarlo dentro de objetos independientes → Los llamaremos **CONSTRUCTORES**



1. **Builder**: Declara los métodos de construcción que deben implementarse para crear partes del producto.
2. **ConcreteBuilder**: Implementa los métodos de construcción para ensamblar y construir partes del producto.
3. **Product**: Representa el objeto complejo que se está construyendo.
4. **Director**: Dirige el proceso de construcción utilizando el builder.
5. **Client**: Crea un builder, lo pasa al director y obtiene el producto resultante.

1. Definir el Producto Complejo

```

class Producto
  attr_accessor :parte_a, :parte_b, :parte_c

  def mostrar_partes
    puts "Partes del Producto: #{[parte_a, parte_b, parte_c].compact.join}
  end
end

```

2. Crear el Builder Abstracto

```
class BuilderAbstracto
  def construir_parte_a; raise NotImplementedError; end
  def construir_parte_b; raise NotImplementedError; end
  def construir_parte_c; raise NotImplementedError; end
  def obtener_producto; raise NotImplementedError; end
end
```

3. Crear el Builder Concreto

```
class BuilderConcreto < BuilderAbstracto
  def initialize
    reiniciar
  end

  def reiniciar
    @producto = Producto.new
  end

  def construir_parte_a
    @producto.parte_a = 'Parte A'
  end

  def construir_parte_b
    @producto.parte_b = 'Parte B'
  end

  def construir_parte_c
    @producto.parte_c = 'Parte C'
  end

  def obtener_producto
    producto = @producto
    reiniciar
    producto
  end
end
```

4. Crear el Director

```
class Director
  def initialize(builder)
    @builder = builder
  end

  def construir_producto_minimo
    @builder.construir_parte_a
  end

  def construir_producto_completo
    @builder.construir_parte_a
    @builder.construir_parte_b
    @builder.construir_parte_c
  end
end
```

5. Código del Cliente

```
def codigo_cliente(director)
  puts 'Producto mínimo estándar:'
  director.construir_producto_minimo
  producto = director.builder.obtener_producto
  producto.mostrar_partes

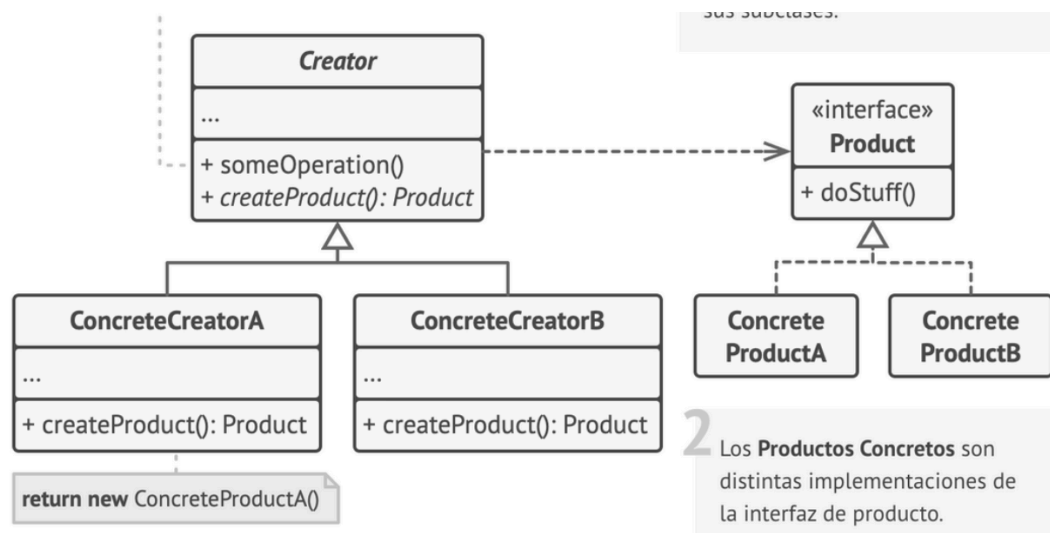
  puts 'Producto completo estándar:'
  director.construir_producto_completo
  producto = director.builder.obtener_producto
  producto.mostrar_partes

  puts 'Producto personalizado:'
  director.builder.construir_parte_a
  director.builder.construir_parte_c
  producto = director.builder.obtener_producto
  producto.mostrar_partes
end
```

```
builder = BuilderConcreto.new
director = Director.new(builder)
codigo_cliente(director)
```

10. FACTORY

- Proporciona una interfaz para crear objetos en una superclase,
- Mientras permite a las subclases alterar el tipo de objetos que se crearán.
- En vez de utilizar "new" para construir objetos, utilizamos el metodo "Fabrica"



1. **Producto (Product):** Declara la interfaz de los objetos que la fábrica crea.
2. **ProductoConcreto (ConcreteProduct):** Implementa la interfaz del producto.
3. **Creador (Creator):** Declara el método factory que devuelve un objeto de tipo Producto.
4. **CreadorConcreto (ConcreteCreator):** Sobrescribe el método factory para devolver instancias de ProductoConcreto.

Ejemplo

1. Definir el Producto Abstracto

```
class Producto
  def operacion
    raise NotImplementedError, "#{self.class} no ha implementado el método"
  end
end
```

2. Definir Productos Concretos

```
class ProductoConcreto1 < Producto
  def operacion
    'Resultado del Producto Concreto 1'
  end
end
```

```
class ProductoConcreto2 < Producto
  def operacion
    'Resultado del Producto Concreto 2'
  end
end
```

3. Definir el Creador Abstracto

```
class Creador
  def factory_method
    raise NotImplementedError, "#{self.class} no ha implementado el método"
  end

  def alguna_operacion
    producto = factory_method
    "Creador: El mismo código del creador acaba de trabajar con #{producto}"
  end
end
```

4. Definir Creadores Concretos

```
class CreadorConcreto1 < Creador
  def factory_method
    ProductoConcreto1.new
  end
end
```

```
class CreadorConcreto2 < Creador
  def factory_method
    ProductoConcreto2.new
  end
end
```

5. Código del Cliente

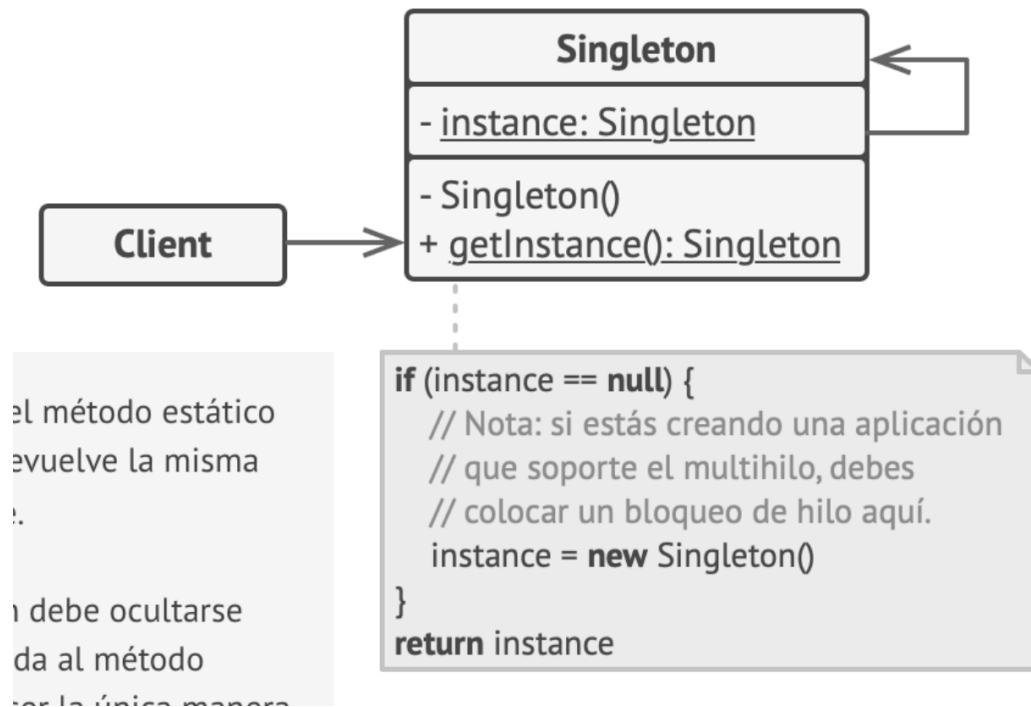
```
def codigo_cliente(creador)
  puts "Cliente: No conozco la clase del creador, pero aún funciona.\n#{creador}"
end
```

```
puts 'App: Lanzado con el CreadorConcreto1.'
codigo_cliente(CreadorConcreto1.new)
```

```
puts "\nApp: Lanzado con el CreadorConcreto2."
codigo_cliente(CreadorConcreto2.new)
```

11. SINGLETON (YO)

- Nos Permite ASEGURAR que una clase solo tenga 1 Instancia
- Proporciona un punto de Acceso Global a Dicha Instancia
- ej: Solo queremos 1 pelota en un partido de fútbol → No permitir una 2da



"El constructor de la clase está oculto para prevenir la creación de más instancias."

```

class Singleton
  @instance = new

  private_class_method :new

  def self.instance
    @instance
  end
end

# Código del Cliente

singleton1 = Singleton.instance
singleton2 = Singleton.instance

if singleton1.equal?(singleton2)
  puts 'Singleton funciona, ambas variables contienen la misma instancia.'
end
  
```

```
else  
  puts 'Singleton falló, las variables contienen diferentes instancias.'  
end
```